Automation in Construction 181 (2026) 106564

Contents lists available at ScienceDirect

Automation in Construction

journal homepage: www.elsevier.com/locate/autcon

Automating digitalization of engineered slopes to support interactive digital twin visualization

Yunhong Lv [a], Yuanen Pang [a], Luyu Ju [a], Weifan Xu [a], Zhenyu Liang [a], Jian He [a], Limin Zhang [a,b,*]

[a] Department of Civil and Environmental Engineering, The Hong Kong University of Science and Technology, HKSAR, China
[b] State Key Laboratory of Climate Resilience for Coastal Cities, The Hong Kong University of Science and Technology, HKSAR, China

ARTICLE INFO

Keywords:
Landslide
Digital twin
Slope safety
Deep learning
BIM
3D reconstruction
Point clouds

ABSTRACT

Slope safety management is transitioning toward the digital twin paradigm, where digital models serve as a base plate for the development of further applications. This paper presents an automated solution to creating digital models for engineered slope digital twins. Utilizing city-scale point cloud, the proposed approach creates lightweight BIM models of engineered slopes consisting of drainage channels and slope topography. The proposed approach leverages a convolutional neural network (CNN) model, DrainNet, for fine-scale drainage channel delineation in images, an automated parametric modeling pipeline for scalable BIM reconstruction, and a web-based digital twin for interactive visualization. Experiments demonstrate that DrainNet achieves state-of-the-art 81.26 % Dice with only 0.28 M parameters on a domain-knowledge-incorporated dataset containing 1226 annotated images. The proposed automated workflow utilizing Grasshopper surpasses the Dynamo workflow in efficiency by 9 times. The BIM models have been integrated into a web-based digital twin platform, facilitating asset management and public engagement.

1. Introduction

Landslides are the most devastating natural hazards in mountainous cities like Hong Kong. Since the 1970s landslide mitigation program, over 60,000 engineered slopes have been registered in Hong Kong [1,2]. However, intensifying rainfall extremes due to climate change challenge the safety of these slopes [3]. Digital twin technology offers transformative potential for landslide risk management by integrating multi-source data, physics-based simulations, and public engagement [4–7]. While existing digital twin implementations demonstrate slope stability analysis capabilities, they are primarily limited to specific sites where detailed models are available [8,9]. This contrasts with the current landslide management needs for detailed modeling of all slopes across urban areas [10–12]. Although Hong Kong mandates Building Information Modeling (BIM) for new slopes [13–15], most existing slopes lack digital representations. A key barrier to digital twin implementation is the resource-intensive creation of accurate three-dimensional (3D) models for a large number of slopes.

This research addresses a critical component of the broader challenge: developing high level-of-detail (LOD) digital models of engineered slopes to enable large-scale digital twin implementation, a prerequisite for advanced landslide risk assessment. The process necessitates both up-to-date digital representations of engineered slopes and bidirectional real world-digital twin connectivity, requiring sustained maintenance efforts [16,17]. Our study focuses on leveraging Hong Kong's open geospatial data to construct foundational slope BIM models that align with engineering standards, serving as digital twin substrates for visualization and analysis. While prior research has automated digital twin creation for infrastructure systems including roads [17,18], bridges [19], and indoor pipelines [20,21], engineered slopes present unique modeling challenges due to irregular geometries and complex components. Notably, no existing studies demonstrate automated generation of foundational digital models for engineered slopes at city-wide risk management scales.

This research employs Hong Kong's territory-wide Light Detection and Ranging (LiDAR) survey data with up to 0.1 m resolution for its superior 3D characterization capabilities compared to traditional imagery and photogrammetry. The modeling scope is first defined by identifying key engineered slope components that require digital twin representation. A typical engineered slope system comprises seven

* Corresponding author at: The Hong Kong University of Science and Technology, Clear Water Bay, Kowloon, Room 3575, Academic Building, Hong Kong, China.
E-mail address: cezhangl@ust.hk (L. Zhang).

https://doi.org/10.1016/j.autcon.2025.106564
Received 11 March 2025; Received in revised form 19 September 2025; Accepted 24 September 2025
Available online 9 October 2025
0926-5805/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

primary elements: topography, drainage channels, catchpits, manholes, soil nails, retaining walls, and vegetation (Fig. 1). Prioritizing topography and drainage channels for modeling in this research addresses both technical constraints and functional imperatives. Two critical factors justify this prioritization. First, the surface topography defines the geometry of a slope model for stability analysis and landslide runout modeling [8,22], while the surface drainage systems directly mediate infiltration and surface erosion, reducing the failure probability of engineered slopes [23]. Evidence from Hong Kong government agencies reveals that many slope failures were caused by drainage system inadequacies, underscoring the operational urgency to optimize channel networks through hydrodynamic simulations. Second, the implementation of a large-scale digital twin introduces significant scalability challenges, with over 60,000 registered slopes requiring digital modeling. Comprehensive modeling of all components would require extraordinary data and computational resources. While all components are equally important for slope modeling, creating accurate representations of topography and surface drainage systems is the essential first step. This approach enables phased integration of additional elements as resources permit while immediately addressing the most pressing needs of Hong Kong's slope management amidst the digital transformation.

This paper proposes a framework for batch-oriented BIM generation of massive engineered slopes. The framework integrates domain-specific datasets, specialized deep learning networks, and fully automated parametric workflows for scalable BIM reconstruction. The resulting models are deployed on a digital twin platform with interactive visualization capabilities, enabling effective risk communication.

The paper is organized as follows: Section 2 reviews digital twin creation processes using point cloud data. Section 3 presents the proposed automated digital twin generation framework. Sections 5–6 detail the methodological workflow. Section 7 discusses results and future directions, with concluding remarks in Section 8.

2. Background

BIM reconstructions of engineering slopes using LiDAR point cloud data involve two critical phases: initial geometric parameter extraction of drainage channels through point cloud segmentation, followed by parametric BIM reconstruction using specialized software. This two-phase method ensures accurate digital representation of structural elements.

2.1. Related research

Accurately identifying object boundaries within point cloud data is typically achieved through two primary methods: (1) directly segmenting 3D point clouds, and (2) mapping 2D segmentation results into 3D space [20]. Deep learning has emerged as a dominant approach for both 2D and 3D semantic segmentation, offering advanced capabilities for object recognition and boundary delineation. These segmentation results can then be utilized in parametric modeling methods to automate the reconstruction of BIM models.

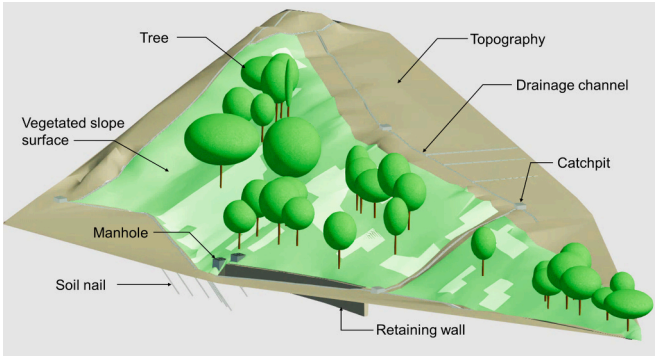

Fig. 1. Components of a typical engineered slope.

2.1.1. 3D Point Clouds Segmentation

While pioneering 3D methods like PointNet [24] introduced permutation-invariant networks through multi-layer perceptrons and max pooling, subsequent advancements such as PointNet++ [25] improved local feature capture through set abstraction. Modern architectures including PointWeb [26] and RandLA-Net [27] address complex scene processing through context-aware aggregation and efficient sampling. Large-scale benchmarks like SemanticKITTI [28] and KITTI360 [29] have significantly advanced data-driven segmentation, with recent supervised learning approaches achieving notable success in multi-object segmentation for indoor and autonomous driving scenarios [30,31]. However, challenges persist in processing large-scale, unstructured point clouds efficiently [21], particularly for extracting slender elements like drainage channels. The alternative 2D projection approach, which converts 3D data into 2D depth maps, leverages mature image segmentation techniques to significantly reduce computational loads compared with direct 3D processing.

2.1.2. 2D Image Segmentation

The evolution of 2D segmentation architectures has progressed through three distinct generations: (1) Fully Convolutional Networks (FCNs) [32] or Convolutional Neural Networks (CNNs) [33–38] excelling in texture pattern recognition, (2) Vision Transformers (ViTs) [39,40] that capture long-range dependencies, and (3) foundation models like Segment Anything Model (SAM) [41,42] that enables zero-shot adaptation for downstream tasks. These methods have been widely applied in tasks related to construction and infrastructure maintenance, such as crack segmentation [43,44], road network segmentation [45,46] and in-door pipeline segmentation [20,21]. To preserve the connectivity of linear objects, some studies focus on refining imperfect road segmentation results to maintain their inherent connectivity [47]. Notably, most existing models are trained on datasets, such as ImageNet [48] for general vision tasks and Chest X-ray8 [49] for medical imaging, which significantly differ from point cloud-derived imagery.

2.1.3. Parametric Modeling

Grasshopper and Dynamo have become pivotal tools in parametric design and cross-platform collaboration within engineering sectors. Their integration of Visual Programming Languages (VPLs) enables automated workflows and complex geometric manipulation [50–54]. These tools seamlessly connect conceptual design with technical execution through computational logic, supporting applications from generative design to data-driven optimization. For instance, Grasshopper facilitates AI-enhanced workflows, such as converting 2D sketches into 3D structural models via deep learning [51], while Dynamo streamlines BIM data interoperability, enabling efficient data transfer between Revit and external platforms like Excel for documentation [54]. Their adaptability extends to advanced computational scenarios, including LiDAR-based point cloud processing for scan to BIM [50,53,55] and reinforcement learning-driven spatial optimization [52]. Custom workflows are necessary to address slope-specific challenges, such as the integration of irregular geometries and domain-specific analysis.

2.2. Research gaps

Despite advancements in segmentation and parametric modeling, significant gaps remain in addressing the unique challenges of batch-oriented BIM reconstruction for engineered slopes. First, computational bottlenecks persist in processing city-scale LiDAR point clouds, where conventional 3D segmentation models struggle to efficiently extract narrow elements like drainage channels from millions of unstructured points. Although 2D projection methods reduce

computational cost, their architectures are not specifically designed for direct application to point cloud-derived depth maps. Second, most segmentation models trained on generic datasets fail to capture the geometric and textural patterns of slope drainage channels, compounded by the absence of domain-specific labeled datasets for this infrastructure type. Third, parametric BIM workflows remain underdeveloped for regional slope systems. While Grasshopper and Dynamo demonstrate potential in scan-to-BIM scenarios [50–56], existing implementations lack automated pipelines to reconcile the geometric complexity of slopes for geohazard management, resulting in fragmented workflows that impede scalable reconstruction. These research gaps collectively highlight the necessity for an integrated framework that combines domain-adapted segmentation, computational efficiency, and automated parametric modeling for engineered slopes.

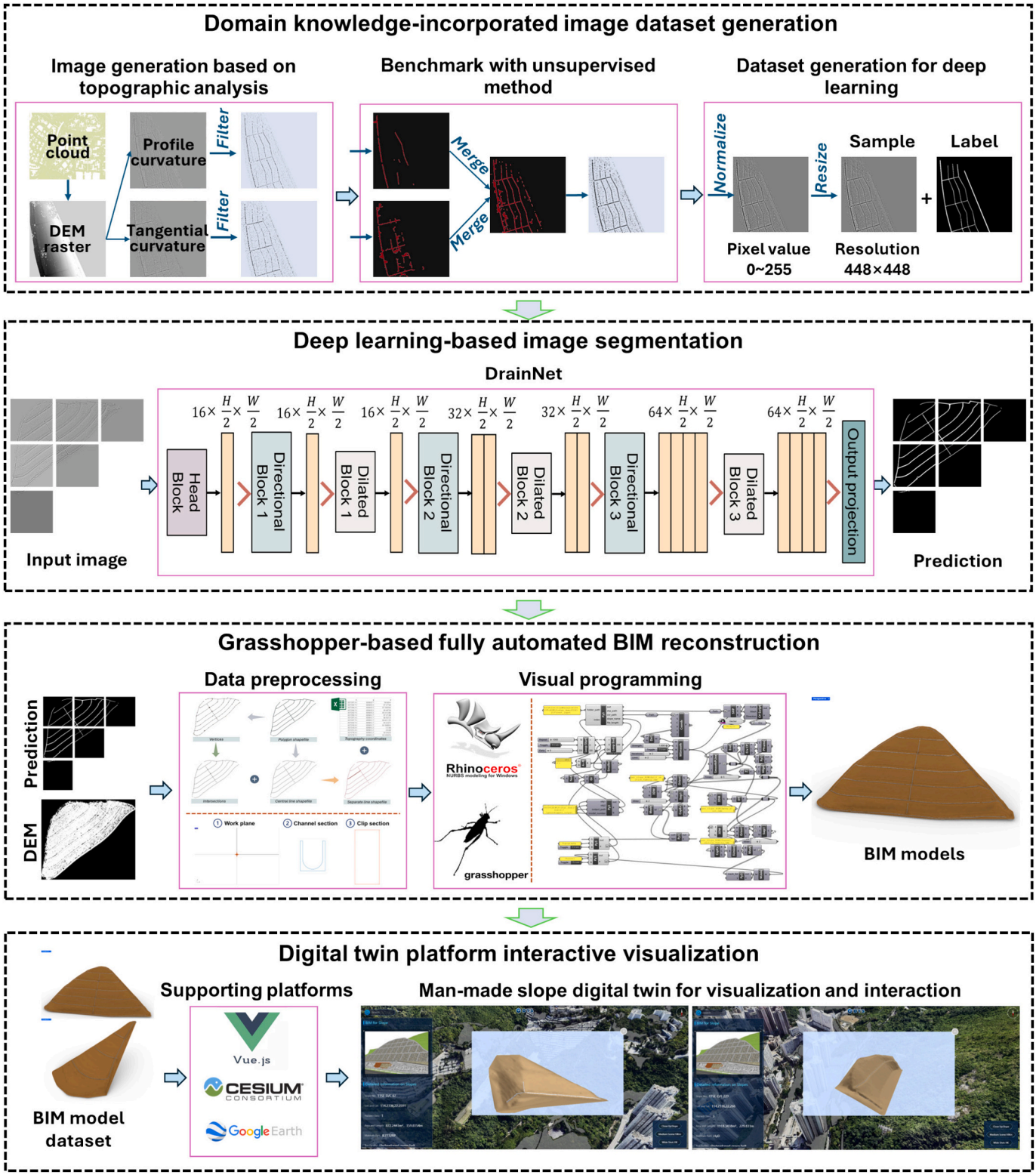

Fig. 2. Overview of the proposed framework for large-scale engineered slope digitalization.

2.3. Research objectives

To bridge the aforementioned research gaps, this paper proposes a framework for batch-oriented BIM generation of engineered slopes. This study aims to:

1. Establish a domain knowledge incorporated dataset comprising labeled images of surface drainage channels on engineered slopes.
2. Develop a specialized deep learning model tailored for slope surface drainage channel segmentation with high efficiency and low computational cost.
3. Design a fully automated parametric workflow to enable efficient and scalable reconstruction of lightweight BIM models that enable scalable digital twin deployment.

3. Methodology

The proposed framework for digitizing massive engineered slopes comprises four principal components (Fig. 2): (1) domain knowledge-incorporated image dataset generation, (2) deep learning-based image segmentation, (3) Grasshopper-based fully automated BIM reconstruction, and (4) digital twin platform interactive visualization.

This study employs an image-based deep learning method to delineate slope surface drainage channels. The initial phase involves generating depth images from LiDAR point clouds, incorporating domain knowledge to optimally distinguish the characteristics of slope surface drainage channels. Slope surface drainage channels are then delineated manually as labels for segmentation.

The segmentation phase implements customized CNN model DrainNet to address the challenge of detecting local linear features while preserving global contextual awareness. DrainNet integrates directional convolution blocks (horizontal and vertical) for local feature extraction, dilated convolution blocks for expanded receptive fields, and channel attention modules for better generalization on small datasets. DrainNet achieves optimal performance in linear structure identification, particularly for slope surface drainage channel delineation.

The BIM reconstruction phase establishes an automated workflow combining Digital Elevation Model (DEM) data with projected 3D channel mappings. A Rhino-Grasshopper visual programming pipeline processes pre-conditioned topography and channel data through customized Python and C# scripts, incorporating parametric channel cross-sections to generate lightweight BIM models.

The final integration phase implements a web-based city-scale digital twin through Vue.js frontend interfacing with Cesium and Google Earth as base maps. Georeferenced BIM models are superimposed on Hong Kong's photogrammetric base maps, enabling interactive 3D visualization with attribute query functionalities for enhanced slope safety management and decision support.

4. Slope Drainage Channel feature dataset generation

3D point cloud data serves as semantic-less raw input for this research. It can be directly converted to 2D DEM rasters to enable deep learning-based image segmentation. In this section, relevant features are extracted from the DEM for drainage channel segmentation, further forming a labeled dataset for deep learning.

4.1. Feature selection

Feature selection is critical for training vision deep learning models, especially when dealing with scenarios that require the creation of a new labeled dataset. In such cases, the availability of labeled samples is often constrained by the substantial labor costs associated with data annotation, making it essential to maximize the utility of the limited data at hand. As depicted in Fig. 3, the majority of drainage channels on engineered slopes are narrower than 0.5 m. Therefore, high-resolution features are essential to accurately capture and segment these narrow channels. The DEM rasters with a resolution of 0.1 m are derived directly

Fig. 3. Drainage channels on engineered slopes.

from the pre-filtered ground point cloud data without any interpolation operation, ensuring data fidelity. Drainage channels on a slope exhibit distinct topographic changes that can be effectively revealed through terrain curvature analysis based on DEM raster. As shown in Fig. 4, two specific types of curvature are particularly relevant: profile curvature, which measures the curvature along the direction of the steepest slope (maximum gradient direction), and tangential curvature, which measures the curvature along the contour direction (perpendicular to the slope direction). These curvatures provide critical information on the geometric characteristics of the terrain surface. Profile curvature k_p and tangential curvature k_t can be calculated as follows [56]:

$$k_p = -\frac{p^2 \bullet r + 2pq \bullet s + q^2 \bullet t}{(p^2+q^2) \bullet \sqrt{1+p^2+q^2}} \quad (1)$$

$$k_t = -\frac{q^2 \bullet r - 2pq \bullet s + p^2 \bullet t}{(p^2+q^2) \bullet \sqrt{1+p^2+q^2}} \quad (2)$$

Where z is the surface elevation; $p = \frac{\partial z}{\partial x}$ and $q = \frac{\partial z}{\partial y}$, representing the slope in x-direction and y-direction, respectively; $r = \frac{\partial^2 z}{\partial x^2}$ and $t = \frac{\partial^2 z}{\partial y^2}$, representing the second derivative of elevation in x-direction and y-direction, respectively; and $s = \frac{\partial^2 z}{\partial x \partial y}$ is the mixed second derivative of elevation.

The two curvatures outlines the slope drainage channels. Specifically, positive curvature values indicate convex topographic changes, such as ridges, while negative values signify concave topographic changes, such as valleys. Drainage channels on slopes typically exhibit negative values (Fig. 4). To enhance the representations of drainage channels, positive values are removed from the curvature images based on domain knowledge. The two curvature images are then summed with a weight factor of 0.5 to create a composite image that better characterizes the channel features in different directions (Fig. 4). Curvature analysis of the DEM raster was carried out with Python's Whitebox and Rasterio libraries.

4.2. Dataset processing

CurvaChannel, a dataset consisting of 1226 images with manual annotations, is established for deep learning-based surface drainage

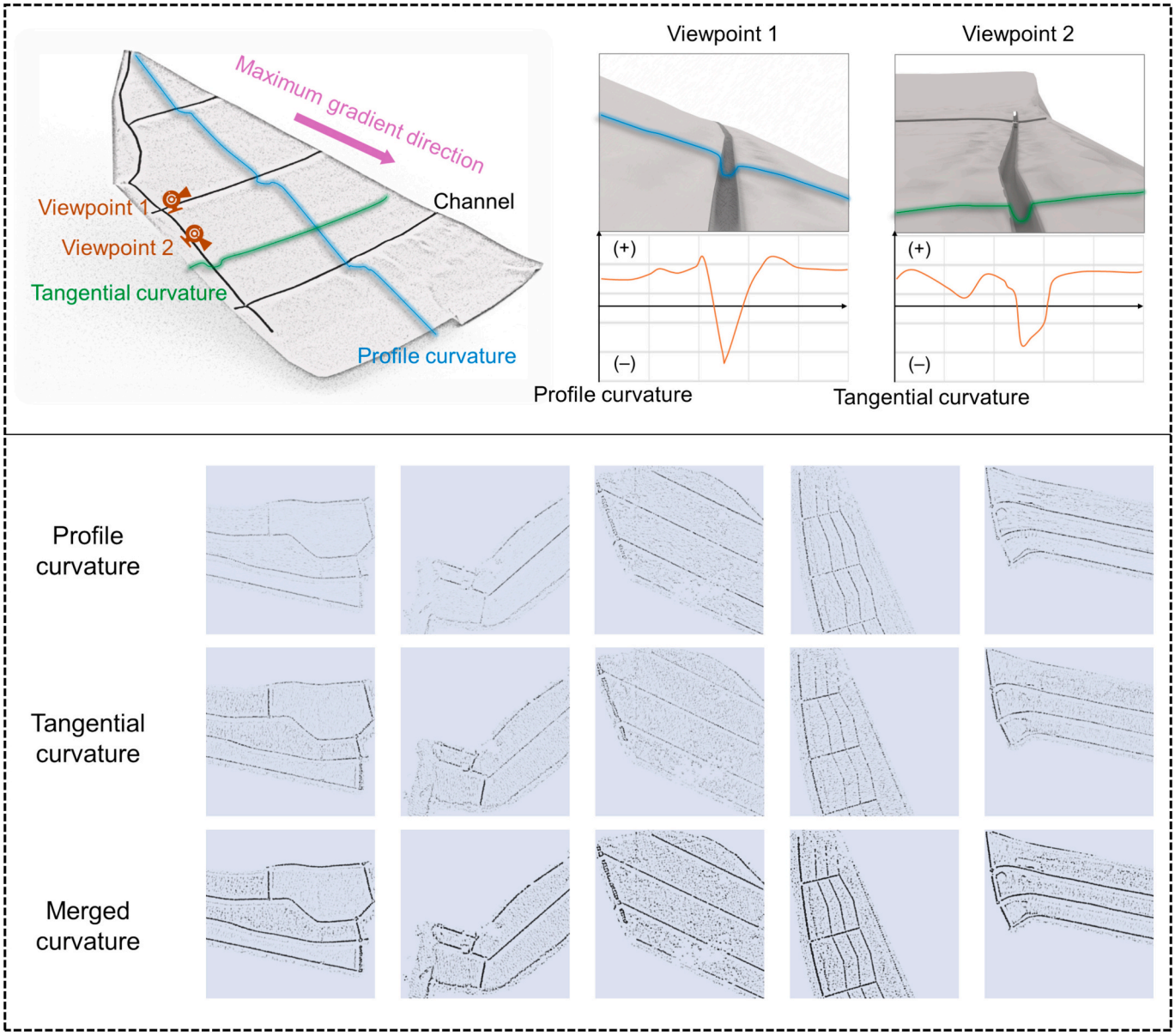

Fig. 4. Curvature image after filtering positive values based on topographic analysis.

channel segmentation. This dataset is derived from a domain-knowledge-driven approach that transforms raw point cloud data into curvature representations. Curvature values are scaled to 0–255 using min-max normalization. All images are resized to 448 × 448 pixels. For those portions of the images smaller than 448 pixels, the remaining areas are filled with zeros, as illustrated in Fig. 5. Data augmentation is not conducted to emphasize the benchmark performance of the models on our dataset and to preserve the correspondence between the curvature directions and the coordinate system for the subsequent BIM reconstruction. The numbers of images for training, validation, and testing are 817, 117, and 234, respectively, as summarized in Table 1.

Table 1
Details of the proposed datasets.

Dataset name	Image size	Number of images for training	Number of images for validation	Number of images for testing
CurvaChannel	448 × 448	817	117	234

5. Deep learning-based image segmentation

A novel CNN model is proposed tailored for segmentation of drainage channels. Additionally, parametric studies are performed on the proposed model to evaluate the contribution of its individual components and validate its overall design.

5.1. Model design

As depicted in Fig. 5, drainage channels on slopes typically exhibit a narrow structure along their axis in imaging contexts, leading to a highly sparse two-dimensional matrix representation. Only a limited number of pixels perpendicular to the axis are relevant for segmentation. The connectivity of drainage channels indicates that if the pixels at both ends are positive samples, the intermediate pixels are also positive, implying that these intermediate pixels can be omitted without affecting the segmentation.

Drawing inspiration from these observations and utilizing 1D convolutions instead of 2D convolutions to simplify model complexity, as implemented in Inception V3 [57], a CNN model named Drainage channel segmentation Network (DrainNet) is proposed for enhanced recognition accuracy under limited dataset conditions. As illustrated in Fig. 6, The model begins with a head block that extracts features from input curvature data. These features are then processed through directional blocks, which employ two perpendicular 1D convolutional layers to focus on the sparse matrix representation of drainage channels, replacing traditional 2D convolutions. Channel attention mechanisms integrate the feature maps from these 1D layers. Next, dilated blocks are introduced incorporating dilated convolutions, enabling rapid expansion of the receptive field without increasing parameters. The head, directional, and dilated block form the encoder of the network, which will be detailed in the following section. Finally, an output projection block acts as the decoder, restoring the spatial resolution of feature maps to their original size and producing the final single-channel output.

5.1.1. Head block

Let the input feature map be denoted as $X_{in} \in R^{C\times H\times W}$. The first step involves passing X through the head block, resulting in the feature map X_h, which is computed as follows:

$$X_h = MaxPool(ReLU(BatchNorm(Conv2D_{3\times 3}(X_{in})))) \tag{3}$$

Here, the max pooling operation reduces the spatial dimensions of the input feature map by half, resulting in $X_h \in R^{C_0\times\frac{H}{2}\times\frac{W}{2}}$. The rectified linear unit (ReLU) activation function introduces non-linearity. Batch normalization is applied to stabilize gradient propagation and reduce overfitting.

5.1.2. Directional block

The directional block copies the input channels into two groups and processes them separately using vertical and horizontal convolutions with kernel size $k \in \{5\times 1, 1\times 5\}$, to capture the orientation-specific characteristics of the feature map X_d,

$$X_d = ReLU(BatchNorm(Conv2D_k(X_{in}))) \tag{4}$$

When $k = 5\times 1$, the vertical directional features X_v are generated; otherwise X_h are generated. These two convolutional operations firstly halve the channel numbers of the feature maps, $X_v, X_h \in R^{\frac{C_0}{2}\times\frac{H}{2}\times\frac{W}{2}}$, then concatenate them to restore the channel numbers to C_0, $X_{cat} \in R^{C_0\times\frac{H}{2}\times\frac{W}{2}}$,

$$X_{cat} = Concat(X_v X_h) \tag{5}$$

Next, the concatenated features are subjected to a channel attention mechanism, designed to highlight the most relevant channels for subsequent processing. This mechanism employs global average pooling and global max pooling to extract the global context and salient features, respectively, resulting in two 1 × 1 feature vectors, A_{avg} and A_{max},

$$A_{avg} = \frac{1}{H_{in}\times W_{in}}\sum_{h=1}^{H_{in}}\sum_{w=1}^{W_{in}} X_{(c,h,w)} \tag{6}$$

$$A_{max} = \max_{1\le h\le H_{in}, 1\le w\le W_{in}} X_{(c,h,w)} \tag{7}$$

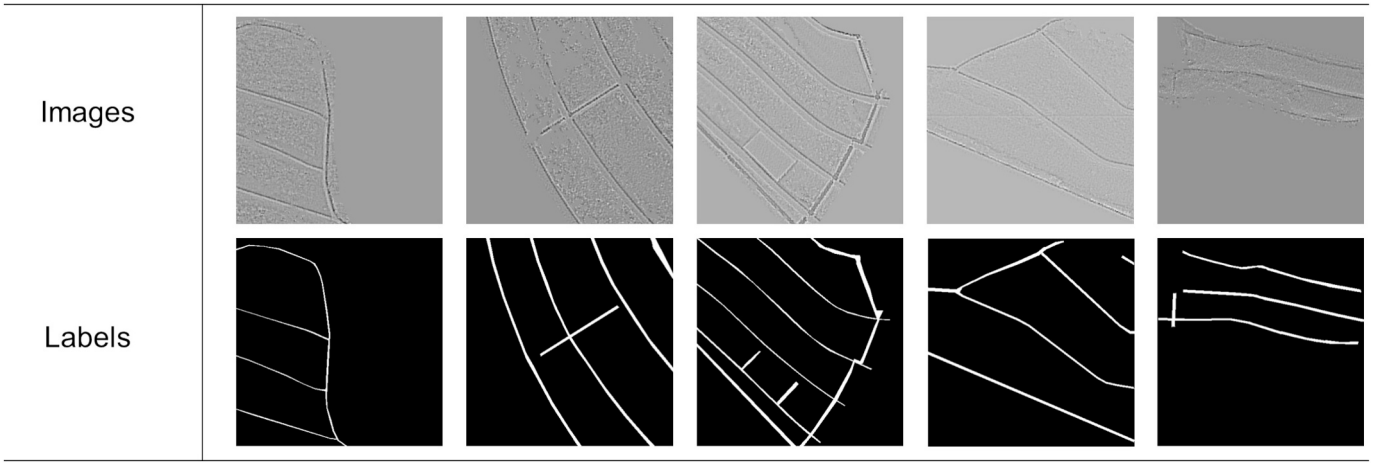

Fig. 5. Image samples of the benchmark dataset, CurvaChannel.

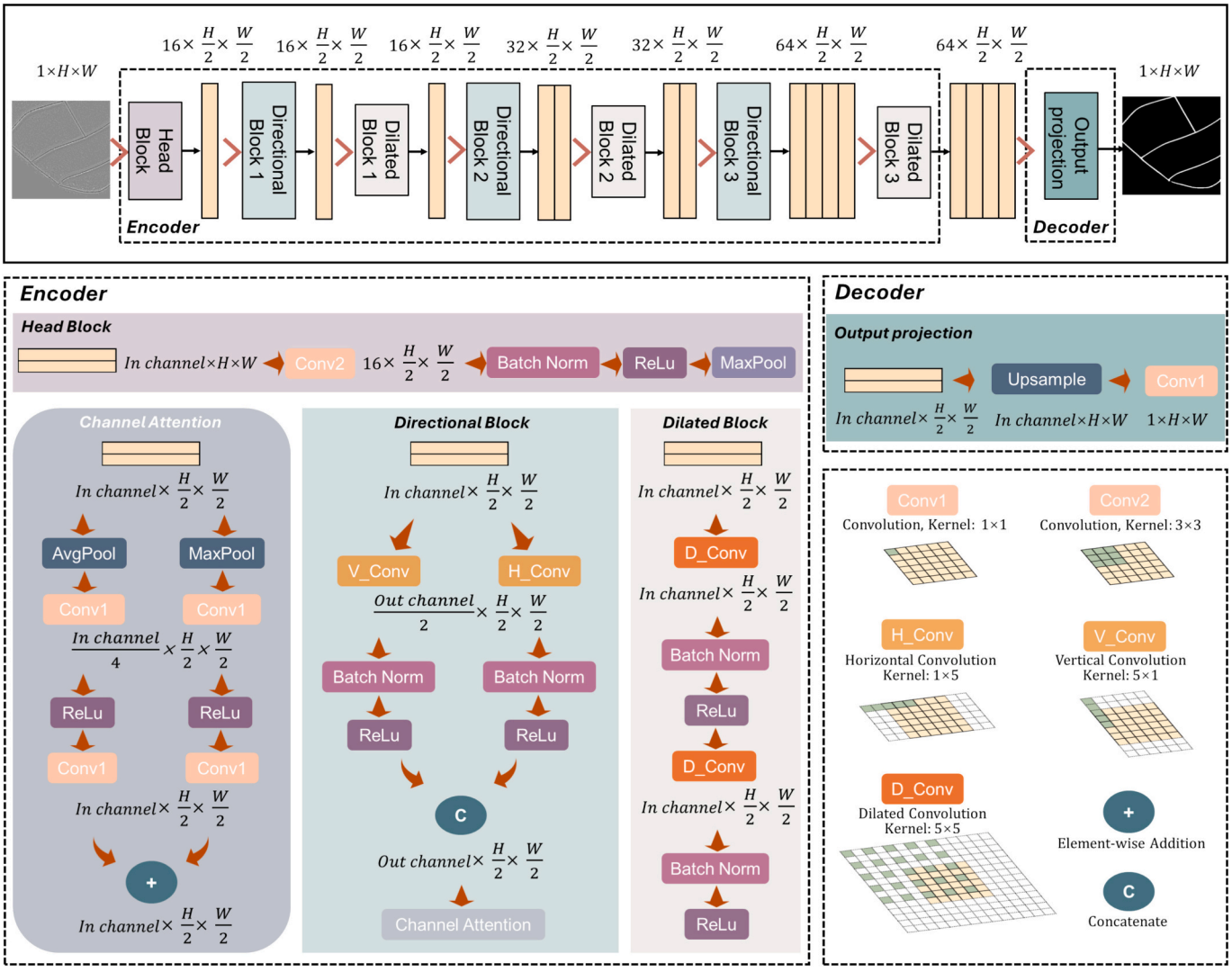

Fig. 6. Model structures of DrainNet.

The above feature vectors are then passed through two shared 1×1 convolutional layer, which forms a fully connected layer to compute the channel attention weights A_C. This process reduces the computational complexity and the number of parameters by compressing the channel dimension:

$$A_C = Conv2D_{1\times1}(ReLU(Conv2D_{1\times1}(X_{in}))) \tag{8}$$

Two channel attention weights, A_C^{avg} and A_C^{max} , are computed. These two attention weights are then element-wise added and passed through a sigmoid activation function to normalize the values to the range [0,1]. Finally, the resulting channel attention weights A are applied to the input features X_{cat} via an element-wise multiplication, achieving adaptive feature enhancement:

$$X_{ca} = X_{cat} \odot A \tag{9}$$

5.1.3. Dilated block

The dilated block utilizes dilated convolutions to expand the receptive field without reducing resolution, integrating both local and global features. Given an input feature map X and a kernel K, a scalar value corresponding to an individual element of the output Y is computed as follows:

$$Y_{c'}(ij) = \sum_{c=1}^{C}\sum_{h=1}^{K_h}\sum_{w=1}^{K_w} K_{c',c}(hw) \bullet X_c(i + r \bullet (h-1)\, j + r \bullet (w-1)) \tag{10}$$

where $K \in R^{C'\times C\times K_h\times K_w}$, $X \in R^{C\times H\times W}$, and $Y \in R^{C'\times H'\times W'}$. Here, c' denotes the output channel index; c represents the input channel index; i and j are the spatial position indices of the output feature map; h and w are the spatial position indices of the convolution kernel; and r is the dilation rate. When $r = 1$, the operation is reduced to a standard convolution. In our network design, the dilated block does not alter the size or the number of channels of the input feature map, ensuring that X and Y share the same dimensions. The output Y is then passed through batch normalization and a ReLU activation function to obtain X_{db}:

$$X_{db} = ReLU(BatchNorm(Y)) \tag{11}$$

where $X_{db} \in R^{C_0\times\frac{H}{2}\times\frac{W}{2}}$.

5.1.4. Feature map transformation

The transformation process of the feature map in the encoder can be summarized as:

$$X_{in}\rightarrow X_h\rightarrow X_{ca}\rightarrow X_{db}\rightarrow X_{ca}\rightarrow X_{db}\rightarrow X_{ca}\rightarrow X_{db} \tag{12}$$

directional blocks and dilated blocks are alternated to extract and

enhance features. The directional block is responsible for capturing detailed directional features of the input through channel-specific operations, with the number of channels $C_0 \in \{16, 32, 64\}$. The dilated block, on the other hand, expands the effective receptive field to capture more contextual information.

5.2. Implementation details

Twelve deep learning models [34,36–40,58–61] including both CNN-based and Transformer-based architecture were trained using our CurvaChannel dataset for comparison. These models were trained with a single Nvidia GeForce RTX 4090D GPU with 24 GB of memory. The AdamW optimizer is employed for training the networks, with hyperparameters set to their default values: $\beta = (0.9, 0.999)$, $\varepsilon = 10^{-8}$, and weight decay = 0.01. Notably, the batch size is configured to 16, and the initial learning rate is 10^{-3}. All models are trained for 60 epochs prior to inference.

In this study, the binary cross-entropy (BCE) with logits loss is used as a loss function to evaluate the discrepancy between the predicted drainage channel mask and the ground truth. The BCE loss function is a common practice in binary segmentation tasks. BCE with logits loss integrates both the sigmoid activation and the BCE loss into a single function, balancing convergence speed, stability, and performance during training. The BCE with logits loss is computed as follows:

$$\mathscr{L}_{BCE\ with\ logits} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log\sigma(\widehat{y}_i) + (1-y_i)log(1-\sigma(\widehat{y}_i))] \tag{13}$$

where y_i represents the ground truth label for the i-th pixel; $\widehat{y}_i$ denotes the predicted logit of the i-th pixel belonging to the drainage channel; $\sigma(\bullet)$ is the sigmoid function; and N is the total number of pixels in the image.

The training and validation loss curves (Fig. 7) reveal distinct behavioral patterns between model families. While CNN-based models generally demonstrate stable convergence, Transformer-based architectures exhibit limitations in capturing drainage channels, as evidenced by the poor performance of ViT and SegFormer(b5). However, PSPNet, DeepLabV3+, and SegNet exhibit validation loss increases after 30–40 epochs, indicating sensitivity to overfitting. This may stem from their deeper architecture or redundant feature fusion mechanisms, which amplify noises in smaller datasets. In contrast, lightweight variants like MobileNetV2-Lite and HarDNet-Lite show more consistent convergence with minimal overfitting, benefiting from their reduced complexity and efficient feature extraction, which align closely with DrainNet's stable patterns. However, their training and validation losses are notably higher than those of larger-parameter models, reflecting a trade-off between efficiency and raw performance capacity. To compare the optimal performance of each model, the parameter combination that yields the minimum validation loss is chosen for assessing model inference capabilities.

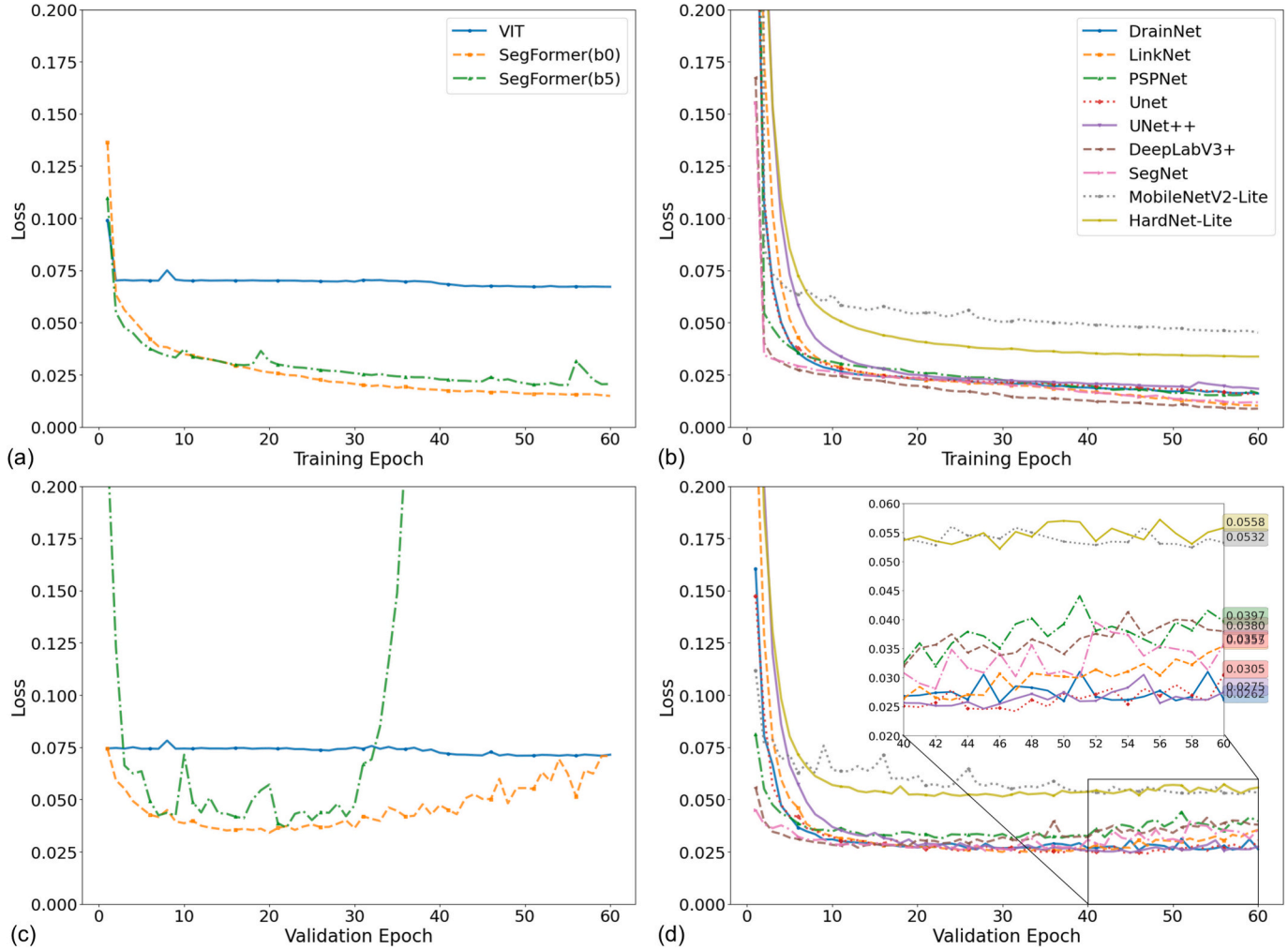

Fig. 7. Training and validation loss: (a) and (c) Transformer-based models; (b) and (d) CNN-based models.

5.3. Validation and comparison

Effectiveness metrics and efficiency metrics collectively provide a comprehensive evaluation of both the effectiveness and efficiency of the proposed model. Intersection over union (IoU), Dice score and skeletonized connectivity error (SCE) are used as metrics to assess the performance of the model. The IoU metric calculates the ratio of the intersection to the union of the predicted and ground truth regions, while the Dice coefficient is a statistic that measures twice the size of the intersection divided by the sum of sizes of the predicted and ground truth regions. The formulas are as follows:

$$IoU = \frac{|GT \cap P|}{|GT \cup P|} \tag{14}$$

$$Dice = \frac{2 \times |GT \cap P|}{|GT| + |P|} \tag{15}$$

where GT represents the area of the ground truth, and P denotes the area of the predicted results. The integration of these metrics allows for a comprehensive assessment of segmentation results. Additionally, to assess structural connectivity, the self-designed metric SCE is included. SCE quantifies topological discrepancies after skeletonization, measuring errors such as disconnections or false merges between predicted and ground truth skeletons. The use of skeletonization to formulate topology-aware metrics has been proven effective in recent literature [62]. Instead of measuring pixel-level overlap, SCE calculates the absolute difference in the number of connected components between the predicted and ground-truth skeletons. This provides a direct and interpretable measure of splitting or merging errors in the segmented network. The formula is as follows:

$$SCE = \frac{|N_{cc}(S(P)) - N_{cc}(S(GT))|}{N_{cc}(S(GT))} \tag{16}$$

where $S(\bullet)$ denotes the skeletonization operator, $N_{cc}(\bullet)$ calculates the number of connected components. A lower SCE indicates better preservation of connectivity.

To evaluate the computational efficiency of the model, the number of parameters (#Params) and multiply-accumulate operations (MACs) are considered. MACs calculates the total number of multiply-accumulate operations required for a single forward pass through the model. These metrics measure the model's complexity and computational requirements, which are crucial for practical applications, especially for large scale applications. Table 2 presents the metrics for the 12 networks on the test set, including IoU, Dice score, MACs, and #Params. The best-performing models within the CNN-based and Transformer-based groups are highlighted in bold. DrainNet stands out as the top performer across all metrics, achieving state-of-the-art results (IoU: 73.5 %, Dice: 81.3 %, SCE: 0.24) with minimal computational overhead (14.39 G MACs, 0.28 M #Params). Under equivalent parameter constraints with lightweight networks like MobileNetV2-Lite and HarDNet-Lite, DrainNet delivers far superior performance, even surpassing larger CNN architectures. Its lightweight design and efficient feature aggregation mechanisms likely contribute to robust generalization. Among the Transformer-based models, SegFormer(b0) achieves the best performance, with an IoU of 62.1 %, a Dice score of 69.0 % and a SCE of 0.54, while requiring only 1.30 G MACs and 3.71 M parameters. This highlights its parameter efficiency compared to larger variants like SegFormer(b5) (84.59 M #Params, 19.12 G MACs). However, their performance remains suboptimal compared to CNN-based counterparts, particularly in scenarios with limited training data. The lack of spatially localized feature extraction mechanisms potentially hinders their adaptability to pixel-level segmentation tasks. This aligns with prior observations that the pure Transformer architecture requires extensive pretraining or large-scale datasets to achieve competitive results.

Fig. 8 shows the segmentation results of the best transformer-based model, our model, and 5 other CNN-based models on the test set. DrainNet demonstrates superior performance in segmentation tasks by achieving better completeness and connectivity compared to other models. For the Transformer based model like SegFormer, its reliance on global self-attention often leads to blurred boundaries in dense prediction tasks. The simplified architecture of DrainNet, leveraging fewer parameters and stronger inductive biases through directional convolutions and channel attention, ensures better generalization on small datasets than other CNN models. This is particularly useful for tasks like drainage channel segmentation, where structures are often elongated and have strong spatial orientations.

Table 2
Quantitative evaluation of the results obtained from different models.

Architecture	Model	IoU (%)	Dice (%)	SCE	MACs (G)	#Params (M)
Transformer-based	ViT	49.37	49.68	1.00	12.06	86
	SegFormer (b0)	62.10	69.03	0.54	1.30	3.71
	SegFormer (b5)	53.10	56.22	0.63	19.12	84.59
CNN-based	U-Net	72.81	80.55	0.27	167.48	31.04
	U-Net++	72.13	79.93	0.49	106.20	9.16
	DeepLabV3+	72.44	80.17	0.27	34.80	40.81
	LinkNet	73.40	81.05	0.32	19.31	13.52
	SegNet	73.45	81.20	0.36	140.73	69.15
	PSPNet	69.15	76.77	0.38	141.41	46.70
	MobileNetV2-Lite	50.00	50.01	0.99	0.29	0.72
	HarDNet-Lite	51.10	52.87	0.91	0.18	0.32
	DrainNet	73.52	81.26	0.24	14.39	0.28

Note: U-Net utilizes feature map channel configurations ranging from 32 to 512. SegFormer(b0) uses 32, 64, 160, and 256 channels, while SegFormer(b5) employs 64, 128, 320, and 512 feature map channels, defining the feature map channel dimensions at different network stages. Lite versions were developed to match DrainNet's parameter scale for fair comparison. HarDNet-Lite reduces channels, growth rates, and layers compared to HarDNet39DS. MobileNetV2-Lite adopts a smaller width multiplier and reduced expansion ratios compared to MobileNetV2.

5.4. Ablation study

To validate the design choices of DrainNet, ablation experiments are conducted on two critical aspects: the model configurations in the encoder and the data combination strategies during training. Table 3 compares the DrainNet performance with varying encoder channel configurations. The baseline channel configuration (16, 32, 64) achieves the best metrics while maintaining minimal computational cost. Expanding the encoder to (16, 32, 64, 128) or (16, 32, 64, 128, 256) results in increased MACs and parameters, yet the performance degrades slightly. Deeper architecture introduces redundant feature interactions, which amplify noise in shallow layers and reduce the model's ability to preserve fine-grained spatial details. Moreover, with limited training data, deeper networks struggle to generalize.

Table 4 details the impact of the channel attention module. Its inclusion yields a consistent improvement across all metrics. This module effectively enhances the model's focus on salient channel features and improves the structural coherence of the segmentation.

Table 5 evaluates the impact of integrating DEM data as an additional channel of the input data. Directly using curvature maps derived from DEM achieves superior performance. When incorporating DEM data, the original input images become 2-channel images, but this degrades segmentation results. Curvature maps explicitly encode drainage channel geometry by incorporating profile and tangential curvature, providing physically meaningful priors that align with the drainage channel segmentation. In contrast, incorporating raw DEM introduces less relevant signals, such as non-channel terrain fluctuations, which dilute the model's focus on channel-specific patterns.

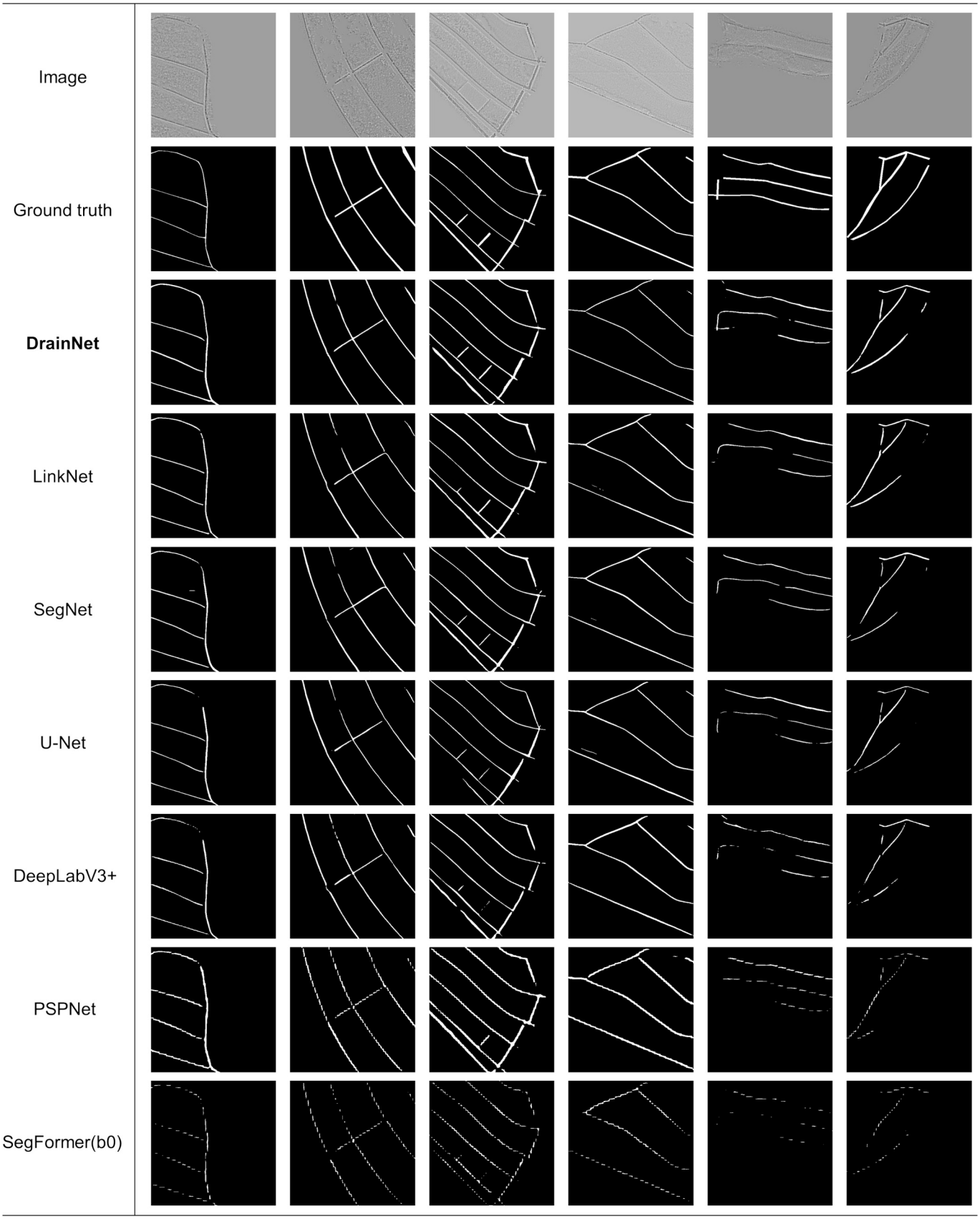

Fig. 8. Drainage channel segmentation results from different deep learning models.

Table 3
Ablation study of the feature map channel configuration in the encoder structure.

Channel configuration	IoU (%)	Dice (%)	SCE	MACs(G)	Param(M)
(16, 32, 64)	73.52	81.26	0.24	14.39	0.28
(16, 32, 64,128)	73.28	80.95	0.26	57.71	1.16
(16, 32, 64,128,256)	73.26	80.89	0.34	230.67	4.63

Table 4
Ablation study of channel attention module in the encoder structure.

Channel attention	IoU (%)	Dice (%)	SCE
✓	73.52	81.26	0.24
X	72.74	80.52	0.28

Table 5
Ablation study of the data combination in model training.

DEM incorporated	IoU (%)	Dice (%)	SCE
X	73.52	81.26	0.24
✓	72.75	80.52	0.36

6. BIM reconstruction and visualization

Using the proposed deep learning model, the 2D distribution of the drainage channels on slope surfaces can be readily obtained. In this section, the 2D channel distribution and topography data will be incorporated into an automated slope BIM model reconstruction workflow to support digital twin visualization and interaction.

6.1. Data Preprocessing

The aim is to construct engineered slope BIM models with detailed drainage channels that adhere to the slope surfaces. In order to handle massive slopes, Grasshopper is used as a parametric modeling tool to automate the BIM reconstruction. The data is converted to certain formats compatible with Grasshopper.

6.1.1. Mapping of segmentation results

The 2D binary segmentation results generated by DrainNet lack geographical information. To accurately place drainage channels on the terrain in Grasshopper, a series of preprocessing steps are applied, as illustrated in Fig. 9.

The original segmentation results are 448 × 448 binary images representing portions of slopes. These images are stitched together to create a complete segmentation map for a slope. Irrelevant small regions or artifacts are removed based on an area threshold. Using OpenCV, the contours of continuous positive pixels were extracted, and the ratio of the contour area to the area of its minimum enclosing circle was calculated. This ratio, empirically determined to range between 0.18 and 0.20, was used to filter non-channel-like noises. A morphological closing operation using OpenCV can be performed to bridge small gaps, using a small kernel size of 3 pixels (0.3 m) to bridge gaps without significantly altering shapes. Alignment with the built environment is verified through overlay on DEM data and visual inspections, though complete validation requires refined ground truth data. The geographic information from the corresponding high-resolution DEM data is copied to the processed binary map. Then binary map is transformed into a polygon shapefile, a widely-used format in Geographic Information System (GIS). The central lines of the drainage channels and vertices of the central lines are extracted. The central lines are divided into individual segments using an intersection-based algorithm to ensure compatibility with Grasshopper. The pseudocode for this algorithm, detailed in Table A1 in Appendix, identifies vertices where randomly created buffer circles intersect with the surrounding lines more than twice. The resulting segmented line shapefile and DEM coordinates are imported into Grasshopper for 3D terrain modeling and channel placement.

6.1.2. Drainage Channel cross-section Modeling

The cross-section of drainage channels follow the guidance in the Hong Kong Slope Information System (SIS) [63], which records the size of drainage channel at different positions of the slope (crest, toe, berm, on slope). The size of these drainage channels varies from 0.2 to 0.4 m. Since the SIS only provides the outer diameter and does not detail internal parameters or cross-sectional profiles, a representative size of 0.3 m is adopted for modeling purposes. The parametric section is defined by three parameters: pipe thickness (a = 0.06 m), inner diameter (b = 0.24 m), and total depth (c = 0.3 m), which are empirically determined and applied uniformly for all drainage channels in this research. As shown in Fig. 10, the parametric channel cross-section is defined in Rhino using the world origin (0,0,0) as the reference point and the x-y plane as the working plane. A clip section is also created to facilitate Boolean operations, enabling terrain excavation in areas occupied by the channel. The spatial relationships among the world origin, channel cross-section, and clipper cross-section are illustrated in the figure. Both the channel and clipper cross-sections are saved as separate Rhino model files, allowing modular integration into the workflow through direct file referencing.

6.2. Workflow design

A task-specific Grasshopper workflow that integrates both commonly available tool nodes and custom-designed code blocks is developed, enabling efficient and automated batch modeling. Fig. 11 illustrates the prepared workflow script. In this workflow, pre-processed DEM coordinate data (describing the terrain) and shapefile data (describing the drainage channel distribution) are fed into a custom-designed Data Import Module (see Table A2 in Appendix), which parses and matches the data based on slope IDs. The module outputs the file paths for terrain coordinate CSV files and drainage channel shapefiles, the number of slopes to be modeled, and the model IDs. Next, the Topography Read Module resamples and smooths the terrain points and converts them into a mesh object. Simultaneously, the Channel Read Module passes the shapefile data to the Projection Module to transfer the 2D channel central lines into 3D curve objects. The Sweep Plane Relocate Module then aligns the sweep operation's working plane to the starting point of each curve. The Initial Plane Read Module retrieves the origin specified during the creation of the channel cross-section and clipper cross-section, which is later used to define the Initial Plane. In the Channel Creation Module, the channel cross-section is first aligned from the Initial Plane to the Sweep Plane, and then a sweep operation is performed along the curve to generate the complete 3D channel geometry. The Mesh Join Module combines the individually created channel models into a unified mesh model. Simultaneously, the Clipper Mesh Creation Module generates a clipping mesh using the same approach, which will be used to modify the terrain.

In the Topography Clip Module, a custom-designed code (see Table A3 in Appendix) is utilized for batch terrain clipping, removing intersecting or overlapping portions of the terrain mesh with the channel geometry. Since Grasshopper models cannot be directly exported, the completed channel and terrain models are baked into Rhino's workspace using the Bake Module (see Table A4 in Appendix). This module assigns different material properties to the models by specifying layer names. The GUIDs of baked models are then passed into the Model Export Module (see Table A5 in Appendix), where the models are combined into a single 3DM file. The output file is named based on the model IDs generated by the Data Import Module. Additionally, the Model Export Module outputs a numerical value in the range of [0,1] to control the step size in the Iteration Module and a Boolean value to indicate whether

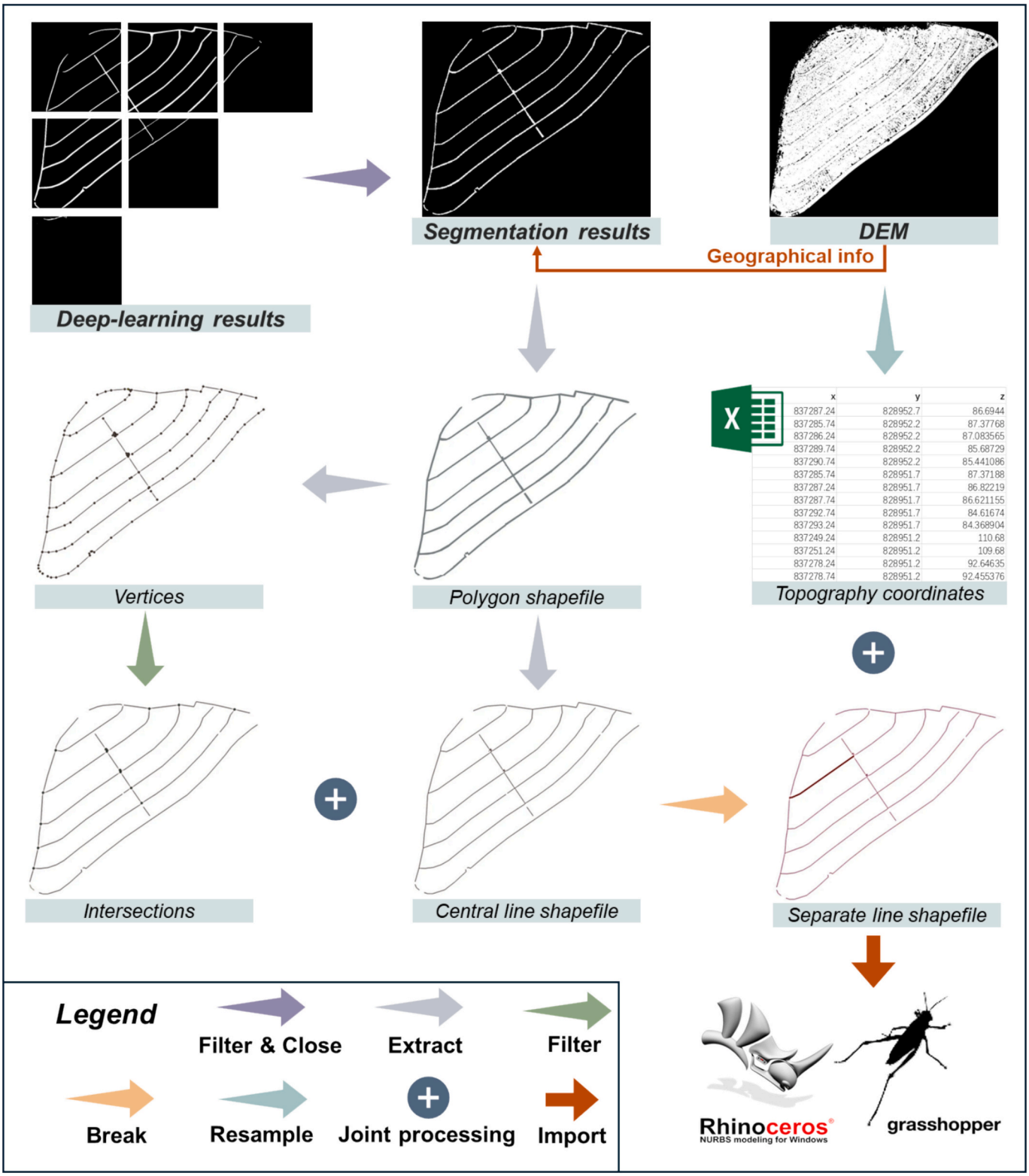

Fig. 9. DEM and drainage channel vector data preprocessing for BIM reconstruction in Grasshopper.

to continue the next iteration. This mechanism ensures that, when processing large batches of files, the workflow can stop iterating if an error occurs in the output files, allowing for easy manual error checking and correction.

6.3. Implementation and comparison

As illustrated in Fig. 12, the BIM model reconstruction process is visualized within Rhino and Grasshopper, showcasing the workflow's key steps. Initially, the input topography and drainage channel data are imported into the Grasshopper workspace. To balance precision and computational efficiency in the BIM reconstruction workflow, the DEM

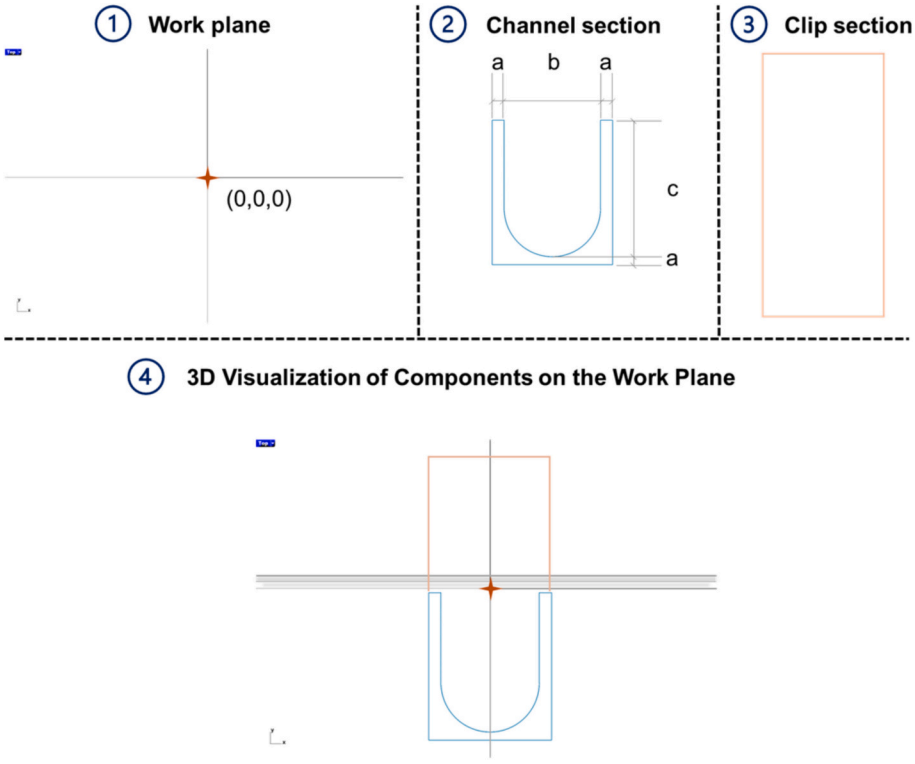

Fig. 10. Creation of parametric cross-sections.

data were resampled to a resolution of 0.5 m to build the topography of engineered slopes. The central lines of the channels are then projected onto the topography, forming a sweep path for the creation of both the channel and the clipper geometries. Grasshopper's Curve.ProjectToMesh method is employed in this step, with a projection precision of 0.1 m to match the resolution of the deep learning segmentation results. The prepared cross-sections, defined in the initial plane, are reoriented to align with the sweep plane, which is determined through the Sweep Plane Locate operation. Using the aligned sections, the channel and clipper geometries are created via the sweep operation. The channel geometry is then directly baked into the Rhino workspace, while the clipper is utilized to modify the topography, resulting in a clipped topography. This clipped topography is subsequently baked into Rhino as well. Finally, the baked channel and baked topography are combined to generate a comprehensive BIM model of an engineered slope, where the spatial relationship between the two elements is accurately maintained. The zoomed-in details further illustrate the precise integration of the channel and terrain, highlighting the effectiveness of the workflow in handling geometric and spatial alignment. Using the workflow, lightweight BIM models of engineered slopes can be efficiently produced in batch.

The Grasshopper workflow is compared with that in Revit-Dynamo on modeling efficiency. The corresponding script for Revit-Dynamo workflow is shown in Fig. A1 in Appendix. It is worth noting that the two software platforms use slightly different logics for constructing BIM models of engineered slopes. For simplicity, the placement of a single drainage channel on the topography using Dynamo VPL is tested, without considering the spatial relationship between the drainage channel and the terrain. The resulting model is presented in Fig. A2 in Appendix, with both overall and detailed views.

The execution times for the two workflows are summarized in Table 6. It takes 15 s in Grasshopper to process a single channel without terrain clipping and model export. By contrast, 136 s is needed for Dynamo to process the same task. Further more, Grasshopper successfully modeled all 38 separate channels on the slope, including terrain clipping and final BIM model export, in approximately 47 s. The gain primarily stems from Grasshopper's multi-channel workflow, which enables parallel processing of tasks such as terrain clipping and BIM model export, distributing computational loads across channels for concurrent execution. Removing the BIM model export step while retaining terrain clipping reduces the execution time to 32 s, and bypassing both terrain clipping and model export further reduces the time to 22 s. Even when modeling complexity increases, Grasshopper maintains a significant speed advantage over Dynamo. Specifically, for the most basic workflow, Grasshopper is approximately nine times faster than Dynamo. The experiment highlights Grasshopper's superior computational efficiency and scalability in parametric BIM workflows, especially when leveraging parallelism for multi-channel operations. This efficiency is primarily attributed to Grasshopper's optimized computational kernel and direct geometric processing, in contrast to Dynamo's heavier integration with Revit's database and API, which introduces additional computational overhead.

6.4. *Interactive visualization in a digital twin platform*

A web-based digital twin platform has been developed for high-quality rendering and real-time data transmission for user interactions. An interactive interface was designed using front-end framework Vue.js to access the model data and attributes stored on the local workstation, enabling real-time interaction with users. Cesium earth and Google earth served as base maps, enhancing demonstrations of complex urban environments. The visualization and interaction of engineered slopes within the digital twin platform are illustrated in Fig. 13.

7. Discussion

This section discusses the failure cases in drainage channel segmentation and the limitations of the proposed framework. By examining these aspects, areas for improvement and future research directions are

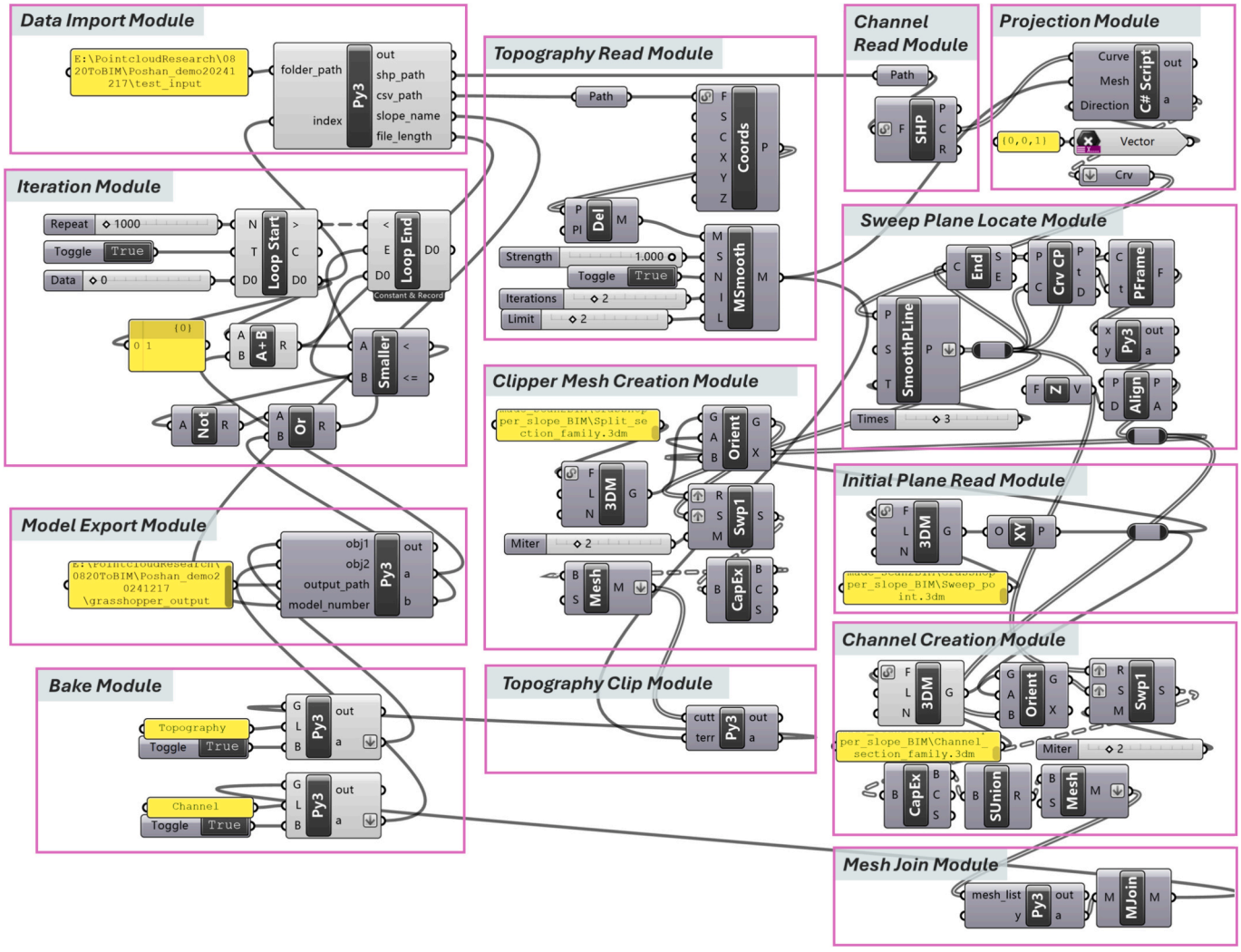

Fig. 11. Prepared script in Grasshopper.

highlighted to enhance the applicability of the proposed approach in slope digitalization and landslide risk management.

7.1. Analysis of failure cases in Drainage Channel delineation

As illustrated in Fig. 14, failure cases in image segmentation fall into two main categories: overestimation and underestimation. Both occur in areas where the point cloud data is sparse or vacant, leading to unexpected topographic variations that disrupt curvature-based analysis. This results in either false positives resembling drainage channels or large blanks in the image. From a model design perspective, such sparsity can amplify noise in feature extraction, causing the network to misinterpret subtle edges or fail to connect fragmented structures.

To address these challenges, a twofold strategy is adopted. First, during dataset annotation, incomplete or discontinuous drainage channels are manually labeled as continuous structures to help the model recognize gaps due to data sparsity (Fig. 14). Second, DrainNet with perpendicular 1D convolutions and channel attention is developed to better capture sparse drainage channel features and improve robustness against incomplete data. Nevertheless, data completeness remains crucial, as it fundamentally affects segmentation accuracy and subsequent modeling.

7.2. Limitations and future work

The method proposed in this research is suitable for drainage channels on slopes that are exposed and uncovered, which can be detected through surface curvature analysis and subsequently segmented using image-based methods. The available data from SIS provides only approximate cross-sectional dimensions, thus empirically determined parameters are applied uniformly for modeling purpose. Using the parametric modeling method, the creation of channel section can be easily generalized when detailed parameters are available. The current segmentation model has limitations in accurately resolving channel width parameters. Alternatively, channel geometry could be derived by converting deep learning segmentation results into shapefiles, though this approach requires further research. Notably, the application of closure operations for maintaining hydraulic connectivity is a simplified way. This post-processing step, while effective, is a heuristic simplification that operates under the assumption that all discontinuities are segmentation errors. Consequently, it may erroneously connect channels that are geomorphologically distinct or introduce non-physical alterations to the channel's shape. The performance of this operation is also contingent upon a relatively complete initial segmentation. Future work should explore more sophisticated approach to establish connectivity, which would more robustly handle complex drainage networks and reduce the reliance on such geometric heuristics.

Furthermore, the current static digital representations of slopes will

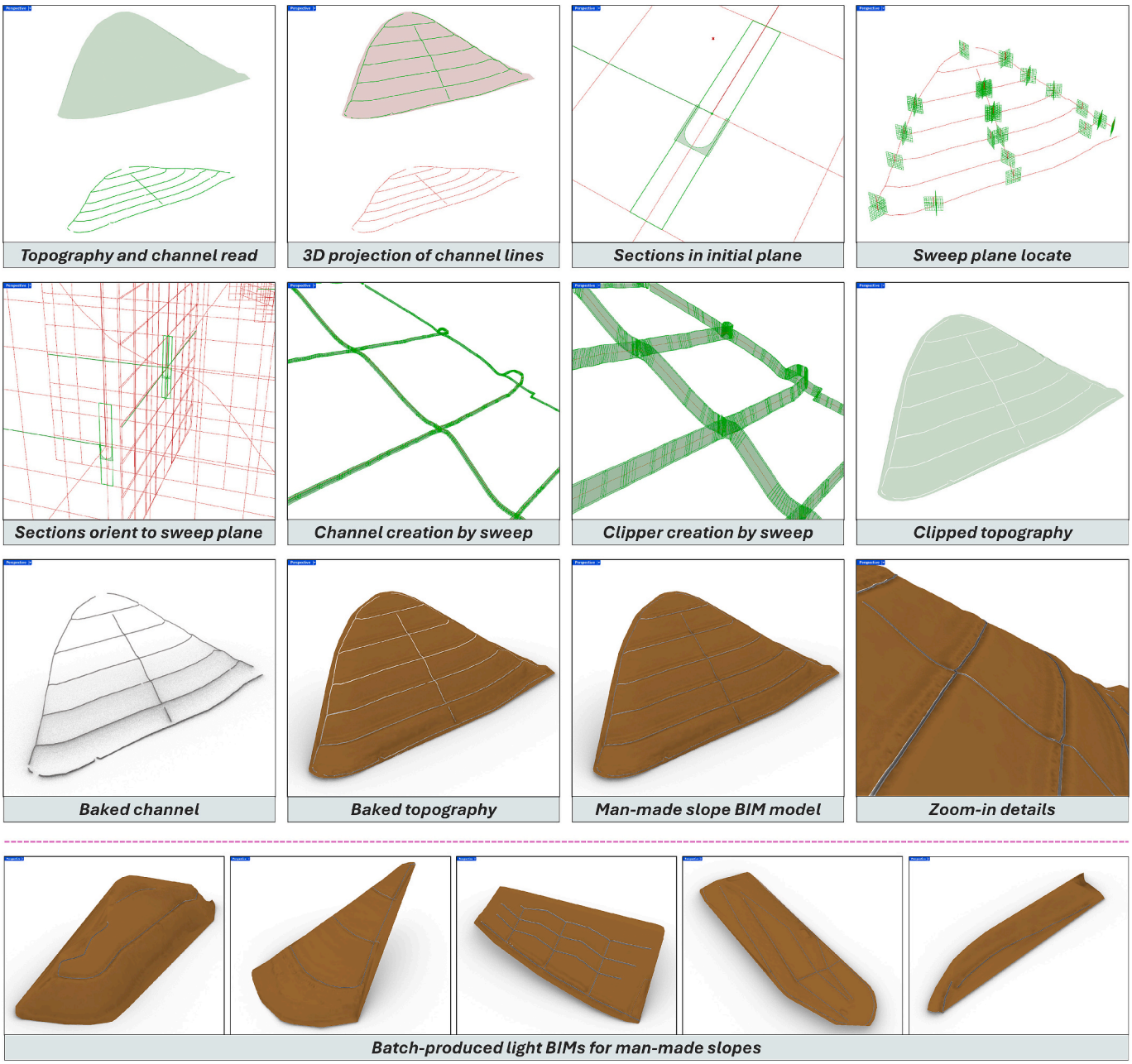

Fig. 12. Creation of engineered slope BIM models in Rhino–Grasshopper.

Table 6
Efficiency comparison between Grasshopper and Dynamo under different settings.*

Tool	Number of channels	Topography clipped	BIM model exported	Running time (s)
Grasshopper	38	✓	✓	47
	38	✓	X	32
	38	X	X	22
	1	X	X	15
Dynamo	1	X	X	136

* Grasshopper is evaluated in both multi-channel (38) and single-channel (1) configurations to highlight efficiency gains from parallelism.

be incorporated with real-time monitoring data to analyze the slope stability. Future research directions will emphasize data-driven susceptibility analysis, hydrodynamic simulations [64], and real-time data analysis [65,66] to enhance landslide risk management. Leveraging detailed drainage information embedded in BIM models and historical landslide records, forthcoming studies will investigate the relationship between drainage channel patterns and slope instability. Comprehensive hydrodynamic simulations will be conducted to optimize drainage channel design, complemented by detailed simulation and visualization of landslide processes. These advancements aim to offer more robust tools for slope stability analysis and risk mitigation strategies.

8. Conclusions

This paper presented an integrated framework for digitalizing engineered slopes to enable advanced visualization and interactive functionalities for landslide risk management. Leveraging high-resolution LiDAR data, the framework systematically digitalizes engineered slopes into BIM representations through drainage channel segmentation, parametric modeling, and web-based platform integration. The major

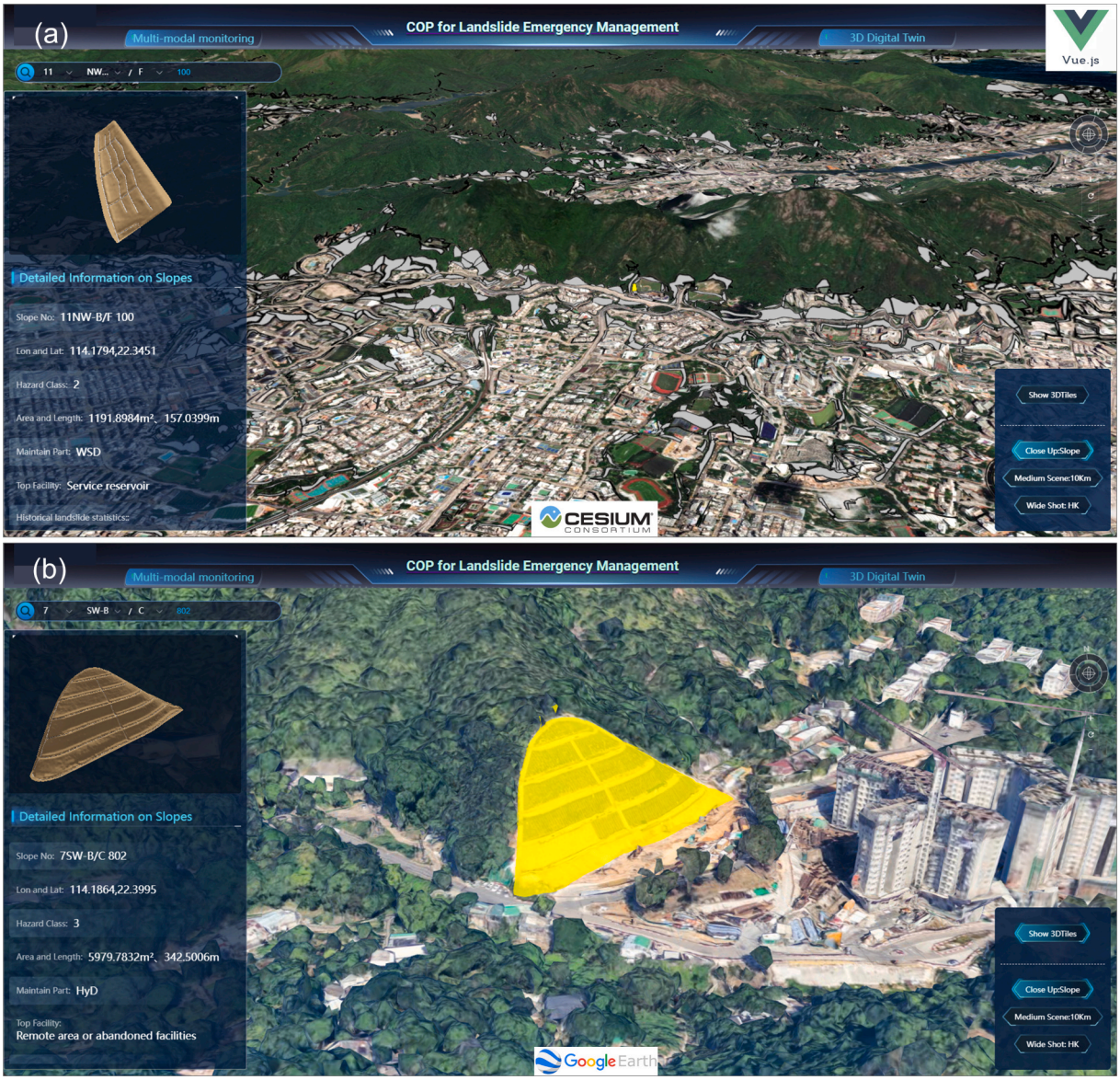

Fig. 13. Engineered slope visualization and interaction in a digital twin platform: (a) city-scale overview; (b) zoom-in view.

contributions of this work are summarized as follows:

1. An image dataset transferred from point cloud data comprising 1226 labeled samples was established, focusing on drainage channels, a critical structural component of engineered slopes. This dataset incorporates domain knowledge in topographic analysis during data annotation (i.e., merged curvature), providing a new perspective and dataset for delineating man-made features on earth surfaces.
2. DrainNet, a CNN architecture optimized for drainage channel segmentation, was proposed. With only 0.28 M parameters, DrainNet achieves state-of-the-art performance (Dice: 81.26 %, IoU: 73.52 %, SCE: 0.24) through innovative integration of directional convolutions, dilated convolutions, and channel attention mechanisms. This design enables precise localization of fine-grained channel structures while maintaining certain topology connectivity and computational efficiency, making it particularly suitable for delineating narrow structures in imaging contexts.
3. A fully automated Grasshopper-based workflow was developed to reconstruct lightweight BIM models of engineered slopes from DEM data and image segmentation results. The workflow integrates task-specific parametric modeling, and efficient batch processing, achieving a modeling speed approximately 9 times faster than Dynamo. This efficiency gain ensures scalable BIM generation for large-scale engineered slope inventories.
4. The BIM models derived were integrated into a web-based digital twin platform to facilitate public participation in landslide risk management. The platform provides immersive 3D visualization and real-time user communications, bridging the gap between technical data and community awareness.

This framework represents a significant shift in geotechnical engineering by transforming raw, semantic-less data into semantically enriched, physically meaningful representations that form the foundation of landslide digital twins. This approach is particularly relevant for managing Hong Kong's extensive inventory of engineered slopes, as it enables scalable and accessible urban landslide risk management through clear digital models. Future research should advance these static models by developing higher LOD BIM and integrating real-time dynamic data from on-site sensors and long-term monitoring, ultimately progressing toward fully realized digital twins. Such developments will require the creation of physics-based and data-driven models to predict slope instability, runout, and associated risks.

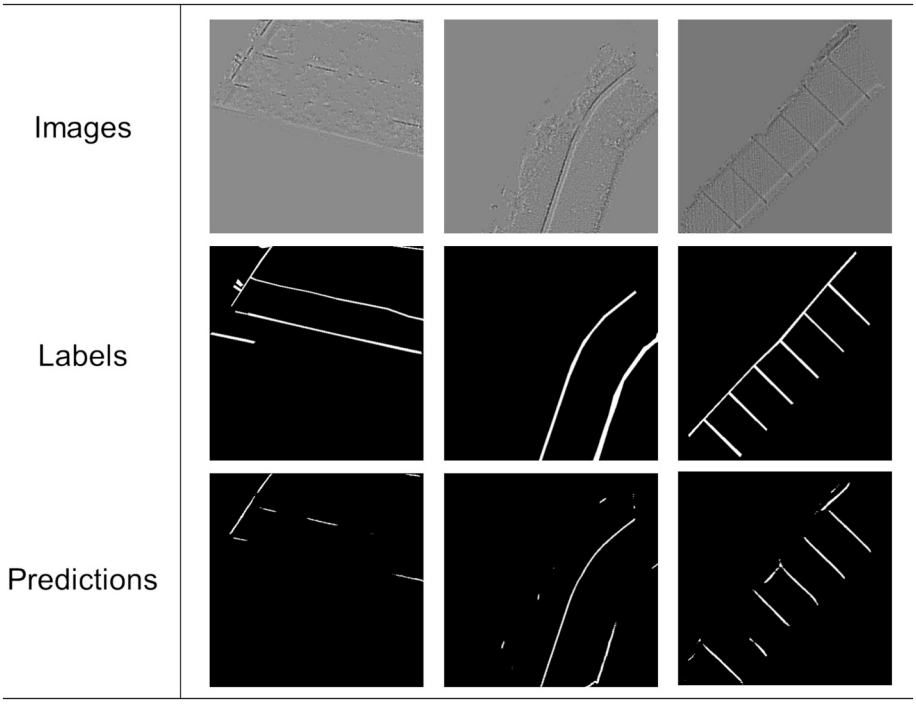

Fig. 14. Representative failure cases in the segmentation of drainage channels.

Furthermore, leveraging open platforms for public engagement will enhance community risk awareness and participation.

Code and data availability

The Pseudo-code for BIM Reconstruction using Grasshopper is available in Appendix A.1. The CurvaChannel dataset is available at https://github.com/Andy-LV-YH/CurvaChannel, the codes for training and testing using DrainNet are also available at https://github.com/Andy-LV-YH/DrainNet.

CRediT authorship contribution statement

Yunhong Lv: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Investigation, Data curation, Conceptualization. **Yuanen Pang:** Writing – review & editing, Validation, Software, Methodology. **Luyu Ju:** Writing – review & editing, Validation, Software, Data curation. **Weifan Xu:** Software, Investigation, Data curation. **Zhenyu Liang:** Writing – review & editing, Software, Investigation. **Jian He:** Writing – review & editing, Visualization. **Limin Zhang:** Writing – review & editing, Supervision, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the Research Grants Council of the Hong Kong SAR Government (Nos. T22-606/23-R), Hetao Shenzhen-Hong Kong Science and Technology Innovation Cooperation Zone (No. HZQB-KCZYB-2020083) and State Key Laboratory of Climate Resilience for Coastal Cities (No. ITC-SKLCRCC26EG01).

Appendix A

A.1. Pseudo-code for BIM Reconstruction using Grasshopper

Table A1
Find intersections algorithm.

```
Data: ISP - Input shapefile path, OSP - output shapefile path
Result: filtered_vertices - Shapefile containing intersection vertices
function find_cross_vertices_in_line (ISP, OSP)
    Load input shapefile as geodataframe (gdf)
    Initialize empty list vertices
    for each geometry in gdf do
        if geometry is LineString then
            Extract all coordinates and add to vertices
        else if geometry is MultiLineString then
            for each sub-line in geometry do
                Extract all coordinates and add to vertices
            end
        end
    end
    Create geodataframe all_vertices_gdf from vertices
    Initialize empty list filtered_vertices
    for each vertex in all_vertices_gdf do
        Create buffer region (unit_circle) with radius 0.1 units around vertex
        Count geometries in gdf intersecting unit_circle (intersection_count)
        if intersection_count ≥ 3 then
            Add vertex to filtered_vertices
        end
    end
    Create geodata frame from filtered_vertices
    Set coordinate reference system to EPSG:2326
    Save filtered_vertices geodata frame to OSP
end
```

Table A2
Data import module.

```
Data: FP - Folder path
Result: [(key, csv_file_path, shp_file_path), file_length]- List of matched file pairs
function match_files(FP):
    Initialize empty dictionaries:
        csv_files = {}
        shp_files = {}
    For each file_name in FP do
        If file_name ends with '.csv' then
            Extract key = file_name without extension
        Else if file_name ends with '.shp' then
            Extract key = file_name without extension
            Add key and full path to shp_files
    Initialize empty list matched_files
    For each key in csv_files do
        If key exists in shp_files then
            Append (key, csv_files[key], shp_files[key]) to matched_files
    Calculate the length of matched_files to file_length
    Append file_length to matched_files
    Return matched_files
```

Table A3
Topography clip module.

```
Data: TM - Initial terrain mesh, SML - splitter mesh list
Result: FM - Accumulative splitting result
function accumulate_split(TM, SML):
    Duplicate TM to FM (to preserve the original mesh)
    For each splitter_guid in SML do
        Try:
            Convert splitter_guid to Rhino.Geometry.Mesh (splitter_mesh)
            If splitter_mesh is valid then
                Perform mesh splitting using Rhino.Geometry.Mesh.Split(FM, splitter_mesh)
                    If split_result has more than one part then
                        Update FM to the first intersected part
                    Else:
                        Print "No valid intersection found. Skipping."
            Else:
                Print "Invalid splitter mesh. Skipping."
        Catch exceptions and print the error details
    If FM is valid then
        Return FM
    Else then
        Return None
```

Table A4
Bake module.

Data: *G* - Geometry to bake, *L* - Layer name for bake, *B* - Bake activation trigger
Result: *a* - The baked geometry's GUID
If *B* is True (bake is activated) **do**
    Find the object reference in the Rhino document using `scriptcontext.doc.Objects.Find(*G*)`
    Extract object *attributes* using `doc_object.Attributes`
    Extract object geometry using `doc_object.Geometry`
    Switch the scriptcontext to the active Rhino document (`scriptcontext.doc = Rhino.RhinoDoc.ActiveDoc`)
    Add *geometry* and *attributes* to the Rhino document using `scriptcontext.doc.Objects.Add(*geometry*, *attributes*)`
    If the layer *L* does not exist in Rhino **then**
        Add the new layer *L*
    Assign the new Rhino object to layer *L* using `rs.ObjectLayer(rhino_brep, *L*)`
    Switch the scriptcontext back to Grasshopper document (`scriptcontext.doc = ghdoc`)
    Set the output *a* to the GUID of the baked geometry
end

Table A5
Model export module.

Data: *OG1, OG2- GUID*s of objects to be exported, *FP*- Path to export the .3dm file (includes the file name), *name* – slope model name, *compress* - Boolean flag to enable or disable file *compress*ion (default: True)
Result: *.3dm file* - Exported at the specified path, optionally clears all objects from the Rhino document, *a* - success or failure flag (1 or 0), *b* - Boolean flag to enable or disable iteration
function export_to_3dm(*OG*, *FP*, *compress*=True):
    Switch to the Rhino main document (`sc.doc = Rhino.RhinoDoc.ActiveDoc`)
    Check if the directory of `*FP*` exists:
        If not then create the directory
    Validate all `*OG*`:
        For each *GUID* in *OG* **do**
            If the *GUID* is invalid **then** raise an exception
    Select the *objects* in `*OG*`
    Configure `write_options`:
        Suppress all warnings
        Disable preview image generation
        Include all document content
        Preserve user data
        If `*compress*` is True **then** enable file *compress*ion
    Write the .3dm file using `sc.doc.Write3dmFile(*FP*, write_options)`
    Deselect all *objects* (`rs.UnselectAllObjects()`)
    Return
---
function clear_all_objects():
    Switch to the Rhino main document (`sc.doc = Rhino.RhinoDoc.ActiveDoc`)
    Use Rhino commands to:
        Select all objects (`_SelAll`)
        Delete selected *objects* (`_Delete`)
    Restore the original Grasshopper document (`sc.doc = current_doc`)
    Return

Main Logic:
Validate Grasshopper inputs:
    Ensure *OG1* is a valid Rhino object
    Ensure *OG2* is a valid Rhino object
    Ensure *FP* is valid
    Ensure *name* is a valid numeric string
Construct the full export path as `os.path.join(*FP*, f"{ *name* }.3dm")`
Try:
    Call `export_to_3dm` with `[ *OG1*, *OG1*]`, ` *FP* `, and `*compress* `
    Call `clear_all_objects`
    Set output `a = 1` (success flag) and `b = False` (continue iteration)
    Catch exceptions and print the error details:
    Set output `a = 0` (failure flag) and `b = True` (stop iteration)
Return

A.2. Dynamo workflow for efficiency comparison of BIM reconstruction

Fig. A1. Script for single drainage channel placement on topography using Dynamo.

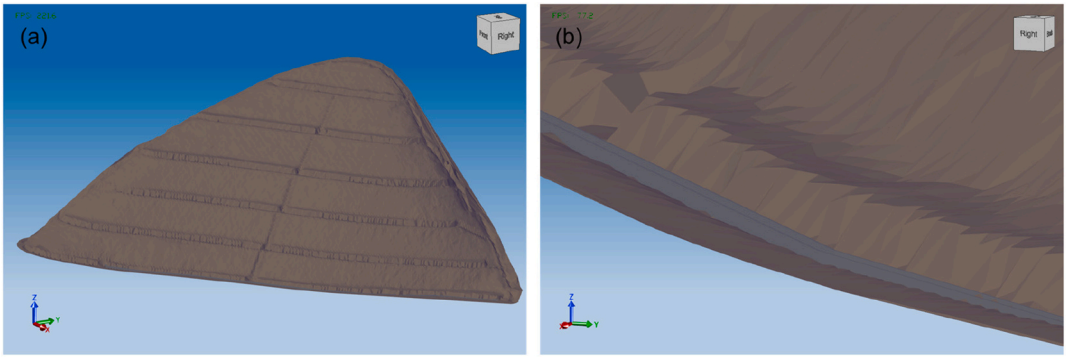

Fig. A2. BIM model of Engineered slope containing single drainage channel using Dynamo: (a) overview; (b) zoom-in view.

Data availability

I have shared the link to my data/code at the Attach File step.

References

[1] Geotechnical Engineering Office, Civil Engineering and Development Department, HKSAR, Slope safety in Hong Kong. https://hkss.cedd.gov.hk/hkss/en/facts-and-figures/slope-information-system/sis/index.html, 2023 accessed February 12, 2025.
[2] H.-Q. Yang, L. Zhang, L. Gao, K.-K. Phoon, X. Wei, On the importance of landslide management: insights from a 32-year database of landslide consequences and rainfall in Hong Kong, Eng. Geol. 299 (2022) 106578, https://doi.org/10.1016/j.enggeo.2022.106578.
[3] Geotechnical Engineering Office, Civil Engineering and Development Department, in: Past notable Landslides, HKSAR, 2023. https://hkss.cedd.gov.hk/hkss/en/facts-and-figures/past-notable-landslides/index.html. accessed February 12, 2025.
[4] Michael Grieves, Digital twin: manufacturing excellence through virtual factory replication, White Paper 1 (2014) 1–7.
[5] P. Voosen, Europe builds 'digital twin' of earth to hone climate forecasts, Science 370 (2020) 16–17, https://doi.org/10.1126/science.370.6512.16.
[6] F. Jiang, L. Ma, T. Broyd, W. Chen, H. Luo, Building digital twins of existing highways using map data based on engineering expertise, Autom. Constr. 134 (2022) 104081, https://doi.org/10.1016/j.autcon.2021.104081.
[7] X. Li, M. Feng, Y. Ran, Y. Su, F. Liu, C. Huang, H. Shen, Q. Xiao, J. Su, S. Yuan, H. Guo, Big data in earth system science and progress towards a digital twin, Nat. Rev. Earth Environ. 4 (2023) 319–332, https://doi.org/10.1038/s43017-023-00409-w.
[8] X. Liu, Y. Wang, R.C.H. Koo, J.S.H. Kwan, Development of a slope digital twin for predicting temporal variation of rainfall-induced slope instability using past slope

performance records and monitoring data, Eng. Geol. 308 (2022) 106825, https://doi.org/10.1016/j.enggeo.2022.106825.
[9] G. Chen, X. Kang, M. Lin, S. Teng, Z. Liu, Stability prediction of soil slopes based on digital twinning and deep learning, Appl. Sci. 13 (2023) 6470, https://doi.org/10.3390/app13116470.
[10] R.W.M. Cheung, Landslide risk management in Hong Kong, Landslides 18 (2021) 3457–3473, https://doi.org/10.1007/s10346-020-01587-0.
[11] T. Xiao, L.-M. Zhang, Data-driven landslide forecasting: methods, data completeness, and real-time warning, Eng. Geol. 317 (2023) 107068, https://doi.org/10.1016/j.enggeo.2023.107068.
[12] J. He, L. Zhang, T. Xiao, H. Wang, H. Luo, Prompt quantitative risk assessment for rain-induced landslides, J. Geotech. Geoenviron. Eng. 149 (2023) 04023023, https://doi.org/10.1061/JGGEFK.GTENG-10980.
[13] S. Azhar, Building information Modeling (BIM): trends, benefits, risks, and challenges for the AEC industry, Leadersh. Manag. Eng. 11 (2011) 241–252, https://doi.org/10.1061/(ASCE)LM.1943-5630.0000127.
[14] Datumate, Decoding the differences: BIM and digital twins explained, 2024. https://www.datumate.com/blog/decoding-the-differences-bim-and-digital-twins/. accessed February 12, 2025.
[15] Construction Industry Council, BIM in HK. https://www.bim.cic.hk/en/bim_in_hk/bim_policy_in_hk, 2025 accessed February 19, 2025.
[16] F. Jiang, L. Ma, T. Broyd, K. Chen, Digital twin and its implementations in the civil engineering sector, Autom. Constr. 130 (2021) 103838, https://doi.org/10.1016/j.autcon.2021.103838.
[17] D. Davletshina, V.K. Reja, I. Brilakis, Automating construction of road digital twin geometry using context and location aware segmentation, Autom. Constr. 168 (2024) 105795, https://doi.org/10.1016/j.autcon.2024.105795.
[18] Y. Pan, M. Wang, L. Lu, R. Wei, S. Cavazzi, M. Peck, I. Brilakis, Scan-to-graph: automatic generation and representation of highway geometric digital twins from point cloud data, Autom. Constr. 166 (2024) 105654, https://doi.org/10.1016/j.autcon.2024.105654.
[19] M. Franciosi, M. Kasser, M. Viviani, Digital twins in bridge engineering for streamlined maintenance and enhanced sustainability, Autom. Constr. 168 (2024) 105834, https://doi.org/10.1016/j.autcon.2024.105834.
[20] B. Wang, Z. Chen, M. Li, Q. Wang, C. Yin, J.C.P. Cheng, Omni-Scan2BIM: a ready-to-use Scan2BIM approach based on vision foundation models for MEP scenes, Autom. Constr. 162 (2024) 105384, https://doi.org/10.1016/j.autcon.2024.105384.
[21] B. Wang, Q. Wang, J.C.P. Cheng, C. Song, C. Yin, Vision-assisted BIM reconstruction from 3D LiDAR point clouds for MEP scenes, Autom. Constr. 133 (2022) 103997, https://doi.org/10.1016/j.autcon.2021.103997.
[22] L.-Y. Ju, T. Xiao, J. He, H.-J. Wang, L.-M. Zhang, Predicting landslide runout paths using terrain matching-targeted machine learning, Eng. Geol. 311 (2022) 106902, https://doi.org/10.1016/j.enggeo.2022.106902.
[23] Geotechnical Engineering Office, Civil Engineering and Development Department, HKSAR, Guide to slope maintenance. https://www.cedd.gov.hk/eng/publications/geo/geoguides/geo-g5/index.html, 2023.
[24] R.Q. Charles, H. Su, M. Kaichun, L.J. Guibas, PointNet: deep learning on point sets for 3D classification and segmentation, in: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 77–85, https://doi.org/10.1109/CVPR.2017.16.
[25] P. Ni, W. Zhang, X. Zhu, Q. Cao, PointNet++ grasping: learning an end-to-end spatial grasp generation algorithm from sparse point clouds, in: 2020 IEEE International Conference on Robotics and Automation (ICRA), 2020, pp. 3619–3625, https://doi.org/10.1109/ICRA40945.2020.9196740.
[26] H. Zhao, L. Jiang, C.-W. Fu, J. Jia, PointWeb: enhancing local neighborhood features for point cloud processing, 2019, pp. 5565–5573, in: https://openaccess.thecvf.com/content_CVPR_2019/html/Zhao_PointWeb_Enhancing_Local_Neighborhood_Features_for_Point_Cloud_Processing_CVPR_2019_paper.html (accessed February 12, 2025).
[27] Q. Hu, B. Yang, L. Xie, S. Rosa, Y. Guo, Z. Wang, N. Trigoni, A. Markham, RandLA-net: efficient semantic segmentation of large-scale point clouds, in: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 11105–11114, https://doi.org/10.1109/CVPR42600.2020.01112.
[28] J. Behley, M. Garbade, A. Milioto, J. Quenzel, S. Behnke, C. Stachniss, J. Gall, SemanticKITTI: A dataset for semantic scene understanding of lidar sequences, 2019, https://doi.org/10.48550/arXiv.1904.01416.
[29] Y. Liao, J. Xie, A. Geiger, KITTI-360: a novel dataset and benchmarks for urban scene understanding in 2D and 3D, IEEE Trans. Pattern Anal. Mach. Intell. 45 (2023) 3292–3310, https://doi.org/10.1109/TPAMI.2022.3179507.
[30] J. Xu, R. Zhang, J. Dou, Y. Zhu, J. Sun, S. Pu, RPVNet: A deep and efficient range-point-voxel fusion network for lidar point cloud segmentation, 2021, https://doi.org/10.48550/arXiv.2103.12978.
[31] D. Robert, B. Vallet, L. Landrieu, Learning multi-view aggregation in the wild for large-scale 3d semantic segmentation, 2022, https://doi.org/10.48550/arXiv.2204.07548.
[32] J. Long, E. Shelhamer, T. Darrell, Fully convolutional networks for semantic segmentation, 2015, https://doi.org/10.48550/arXiv.1411.4038.
[33] A. Krizhevsky, I. Sutskever, G.E. Hinton, ImageNet Classification with Deep Convolutional Neural Networks, in: Advances in Neural Information Processing Systems, Curran Associates, Inc., 2012, in: https://proceedings.neurips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html. accessed February 12, 2025.
[34] O. Ronneberger, P. Fischer, T. Brox, U-Net: Convolutional networks for biomedical image segmentation, 2015, https://doi.org/10.48550/arXiv.1505.04597.
[35] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778, https://doi.org/10.1109/CVPR.2016.90.
[36] A. Chaurasia, E. Culurciello, LinkNet: exploiting encoder representations for efficient semantic segmentation, in: 2017 IEEE Visual Communications and Image Processing (VCIP), 2017, pp. 1–4, https://doi.org/10.1109/VCIP.2017.8305148.
[37] V. Badrinarayanan, A. Kendall, R. Cipolla, SegNet: a deep convolutional encoder-decoder architecture for image segmentation, IEEE Trans. Pattern Anal. Mach. Intell. 39 (2017) 2481–2495, https://doi.org/10.1109/TPAMI.2016.2644615.
[38] Z. Zhou, M.M.R. Siddiquee, N. Tajbakhsh, J. Liang, UNet++: A nested U-Net architecture for medical image segmentation, 2018, https://doi.org/10.48550/arXiv.1807.10165.
[39] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby, An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale. http://arxiv.org/abs/2010.11929, 2021.
[40] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J.M. Alvarez, P. Luo, SegFormer: Simple and efficient design for semantic segmentation with transformers, 2021, https://doi.org/10.48550/arXiv.2105.15203.
[41] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A.C. Berg, W.-Y. Lo, P. Dollár, R. Girshick, Segment anything, in: 2023 IEEE/CVF International Conference on Computer Vision (ICCV), IEEE, Paris, France, 2023, pp. 3992–4003, https://doi.org/10.1109/ICCV51070.2023.00371.
[42] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, C. Feichtenhofer, SAM 2: Segment anything in images and videos, 2024, https://doi.org/10.48550/arXiv.2408.00714.
[43] C. Han, H. Yang, T. Ma, S. Wang, C. Zhao, Y. Yang, CrackDiffusion: a two-stage semantic segmentation framework for pavement crack combining unsupervised and supervised processes, Autom. Constr. 160 (2024) 105332, https://doi.org/10.1016/j.autcon.2024.105332.
[44] Z. He, W. Chen, J. Zhang, Y.-H. Wang, Crack segmentation on steel structures using boundary guidance model, Autom. Constr. 162 (2024) 105354, https://doi.org/10.1016/j.autcon.2024.105354.
[45] R. Lian, L. Huang, Weakly supervised road segmentation in high-resolution remote sensing images using point annotations, IEEE Trans. Geosci. Remote Sensing 60 (2022) 1–13, https://doi.org/10.1109/TGRS.2021.3059088.
[46] H. Liu, X. Zhou, C. Wang, S. Chen, H. Kong, Fourier-deformable convolution network for road segmentation from remote sensing images, IEEE Trans. Geosci. Remote Sensing 62 (2024) 1–17, https://doi.org/10.1109/TGRS.2024.3476087.
[47] L. Han, L. Hou, X. Zheng, Z. Ding, H. Yang, K. Zheng, Segmentation is not the end of road extraction: an all-visible denoising autoencoder for connected and smooth road reconstruction, IEEE Trans. Geosci. Remote Sensing 61 (2023) 1–18, https://doi.org/10.1109/TGRS.2023.3276591.
[48] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei, ImageNet: a large-scale hierarchical image database, in: 2009 IEEE Conference on Computer Vision and Pattern Recognition, 2009, pp. 248–255, https://doi.org/10.1109/CVPR.2009.5206848.
[49] X. Wang, Y. Peng, L. Lu, Z. Lu, M. Bagheri, R.M. Summers, ChestX-Ray8: hospital-scale chest X-ray database and benchmarks on weakly-supervised classification and localization of common thorax diseases, in: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 3462–3471, https://doi.org/10.1109/CVPR.2017.369.
[50] B. Wang, C. Yin, H. Luo, J.C.P. Cheng, Q. Wang, Fully automated generation of parametric BIM for MEP scenes based on terrestrial laser scanning data, Autom. Constr. 125 (2021) 103615, https://doi.org/10.1016/j.autcon.2021.103615.
[51] A. Agirbas, Semi-automated creation of reciprocal frame structures using deep learning, Autom. Constr. 165 (2024) 105515, https://doi.org/10.1016/j.autcon.2024.105515.
[52] Z. Zhang, Z. Guo, H. Zheng, Z. Li, P.F. Yuan, Automated architectural spatial composition via multi-agent deep reinforcement learning for building renovation, Autom. Constr. 167 (2024) 105702, https://doi.org/10.1016/j.autcon.2024.105702.
[53] M. Pepe, A.R. Garofalo, D. Costantino, F.F. Tana, D. Palumbo, V.S. Alfio, E. Spacone, From point cloud to BIM: a new method based on efficient point cloud simplification by geometric feature analysis and building parametric objects in rhinoceros grasshopper software, Remote Sens 16 (2024) 1630, https://doi.org/10.3390/rs16091630.
[54] Z. He, Y.-H. Wang, J. Zhang, Generative AIBIM: an automatic and intelligent structural design pipeline integrating BIM and generative AI, Information Fusion 114 (2025) 102654, https://doi.org/10.1016/j.inffus.2024.102654.
[55] Z. Liang, L. Yang, Z. Liang, J.C.F. Chan, Z. Zhang, M. Wang, J.C.P. Cheng, Optimized UAV view planning for high-quality 3D reconstruction of buildings using a modified sparrow search algorithm, Adv. Eng. Inform. 65 (2025) 103344, https://doi.org/10.1016/j.aei.2025.103344.
[56] Surfer, Profile Curvature and Tangential Curvature. https://surferhelp.goldensoftware.com/gridops/profile_curvature.htm, 2025 accessed February 21, 2025.
[57] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, Z. Wojna, Rethinking the inception architecture for computer vision, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 2818–2826, https://doi.org/10.1109/CVPR.2016.308.
[58] H. Zhao, J. Shi, X. Qi, X. Wang, J. Jia, Pyramid scene parsing network, 2017, https://doi.org/10.48550/arXiv.1612.01105.
[59] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, A.L. Yuille, DeepLab: semantic image segmentation with deep convolutional nets, atrous convolution, and fully

connected CRFs, IEEE Trans. Pattern Anal. Mach. Intell. 40 (2018) 834–848, https://doi.org/10.1109/TPAMI.2017.2699184.
[60] M. Tan, Q.V. Le, EfficientNetV2: Smaller Models and Faster Training, 2021, https://doi.org/10.48550/arXiv.2104.00298.
[61] P. Chao, C.-Y. Kao, Y.-S. Ruan, C.-H. Huang, Y.-L. Lin, HarDNet: A Low Memory Traffic Network, 2019, https://doi.org/10.48550/arXiv.1909.00948.
[62] S. Shit, J.C. Paetzold, A. Sekuboyina, I. Ezhov, A. Unger, A. Zhylka, J.P.W. Pluim, U. Bauer, B.H., Menze, clDice - a novel topology-preserving loss function for tubular structure segmentation, in: In: 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), IEEE, Nashville, TN, USA, 2021, pp. 16555–16564, https://doi.org/10.1109/cvpr46437.2021.01629.
[63] Geotechnical Engineering Office, Civil Engineering and Development Department, HKSAR, Slope information system, (2023). https://hkss.cedd.gov.hk/hkss/en/facts-and-figures/slope-information-system/sis/index.html (accessed February 21, 2025).
[64] S. Kadia, I.A.S. Larsson, M. Billstein, L. Lia, E. Pummer, Experimental and numerical investigations of the water surface profile and wave Extrema of supercritical flows in a narrow channel bend, Sci. Rep. 14 (2024) 12247, https://doi.org/10.1038/s41598-024-61297-8.
[65] L. Wang, H. Liu, F. Zhang, L. Guo, Z. Chen, Spatial structure digital twins: application in intelligent health monitoring of cable dome structures, Autom. Constr. 165 (2024) 105489, https://doi.org/10.1016/j.autcon.2024.105489.
[66] J. Tian, R. Yu, J. Chen, C. Chen, Y. Li, X. Sun, H. Huang, Digital twin model for analyzing deformation and seepage in high earth-rock dams, Autom. Constr. 173 (2025) 106079, https://doi.org/10.1016/j.autcon.2025.106079.